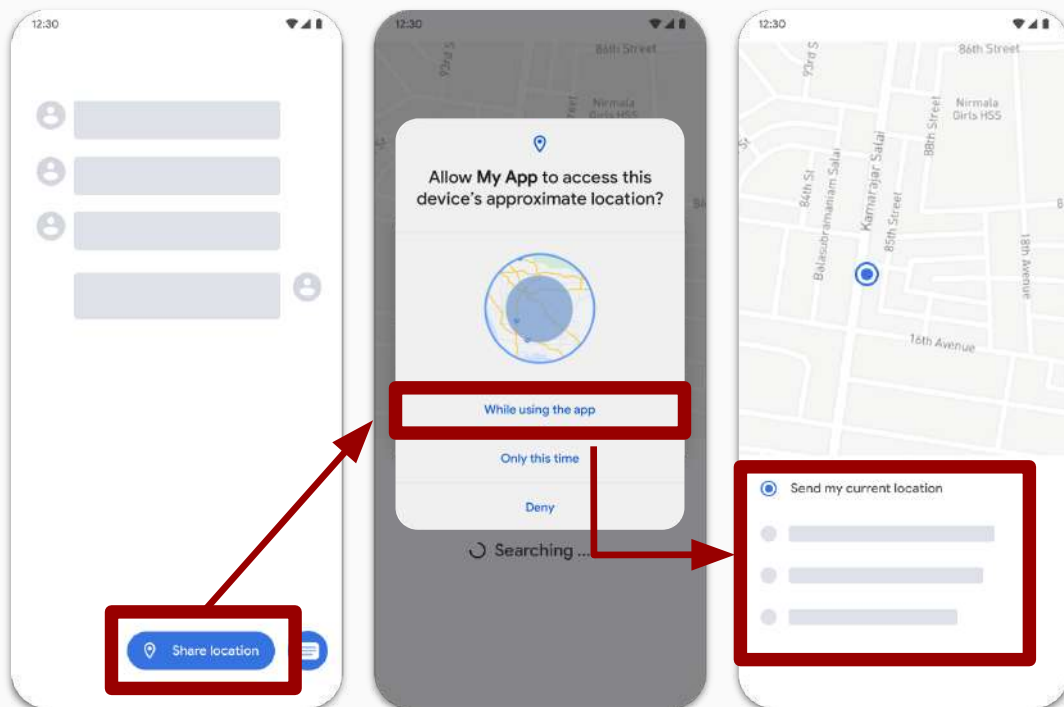




UD6 - Permisos. Sensores. Servicios web. Geolocalización y mapas



Q <https://developer.android.com/static/images/guide/fragments/fragment-screen-sizes.png?hl=es-419>.

Función Compartir ubicación que requiere acceso a la ubicación en primer plano. La función está habilitada si el usuario selecciona Permitir solo con la app en uso.

(Google Android Developers)

I - Permisos



Caso práctico



Q [cottonbro \(https://www.pexels.com/es-es/foto/mano-usb-tecnologia-sujetando-5474298/\)](https://www.pexels.com/es-es/foto/mano-usb-tecnologia-sujetando-5474298/) (Pexels)

- ¡Tenemos un nuevo reto! -informa Ada a su equipo-. Uno de nuestros clientes solicita que una de las aplicaciones Android que le desarrollemos sea capaz de tomar fotografías desde la cámara del dispositivo de los usuarios. ¿Qué problema veis?

- Es una acción es muy intrusiva desde el punto de vista de la privacidad de los usuarios de las aplicaciones -responde Pedro-, ya que podría capturar imágenes e incluso vídeos en cualquier momento. Los

usuarios deben tener todo el control sobre estas acciones.

- Buena puntualización -señala Ada-. Existen otras muchas acciones consideradas peligrosas desde el punto de vista de la privacidad, ya que el teléfono móvil es un acompañante de todas las personas con muchos sensores capaces de captar información visual, audios, posicionamiento mediante GPS, etc., conectado a Internet.

- Si bien es cierto -prosigue Ada en su explicación-, el sistema Android mejora en cada paso de API para ofrecer mayores herramientas de control a los usuarios de sus aplicaciones, ofreciendo distintos permisos agrupados en varias categorías dependiendo de la criticidad de los datos o acciones que protejan.

- Propongo indagar los tipos de permisos que ofrece la API y lo que protegen -lanza María como propuesta, la cual es aceptada por todo el equipo de muy buen grado.

Es imprescindible que los desarrolladores de aplicaciones Android conozcan los distintos tipos de permisos, lo que protegen y, sobre todo, que sigan una serie de pautas y buenas prácticas para que los usuarios de sus aplicaciones estén tranquilos usándolas en conciencia de lo que estas pueden hacer sobre sus dispositivos.



Q [Ministerio de Educación y Formación Profesional.](https://www.minedu.es/)

(Dominio público)

de Educación y Formación Profesional.

 [Aviso Legal](#) 

1.- Análisis sobre los permisos

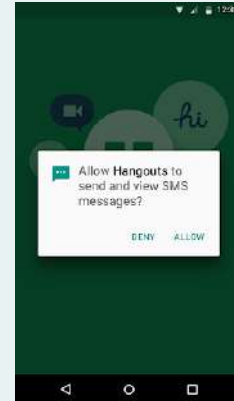


Caso práctico

- Hay tres cosas muy importantes que debemos tener en cuenta cuando hablamos de permisos -explica Ada a su equipo, mostrándoles una pizarra con la información que les quiere transmitir. Quiere que los programadores sean conscientes de cómo deben gestionar los permisos en su aplicación.

- No todos los datos ni todas las acciones accesibles desde un dispositivo Android son iguales desde el punto de vista de la privacidad. No es lo mismo grabar una conversación privada de nuestro usuario, que acceder a un documento que sea ha generado desde la propia aplicación.
- Solicitar permisos de acceso a determinados datos o acciones puede provocar que los usuarios sean reacios a usar nuestra aplicación e incluso lleguen a desinstalarla.
- Es importante minimizar, accediendo a la menor cantidad posible de datos y de acciones en el dispositivo e intentando averiguar la información que nuestra aplicación necesita por otros medios que no impliquen estos permisos. Por ejemplo podemos solicitar el CP del domicilio o lugar de trabajo de nuestro usuario en lugar de acceder a su ubicación para ofrecerle sugerencias cercanas a él.

- El primer paso para construir aplicaciones eficientes y confiables para los usuarios -indica Ada al equipo, que sigue su explicación sin perder detalle-, es conocer los distintos tipos de permisos y sus grupos, para saber cómo Android va a solicitar al usuario el acceso a los mismos.



[Google Developers \(https://developer.android.com/codelabs/android-navigation?hl=es-419#2\)](https://developer.android.com/codelabs/android-navigation?hl=es-419#2) (Uso educativo nc)

Los permisos de las aplicaciones Android ayudan a respetar la privacidad del usuario, ya que protegen el acceso:

- Datos restringidos, como el estado del sistema y la información de contacto del usuario.
- Acciones restringidas, como conectarte a un dispositivo vinculado y grabar audios.

En el caso de que tu aplicación necesite acceder a datos o acciones restringidas a través de un permiso, debes mencionarlo en el archivo de manifiesto AndroidManifest.xml.

Según el grado de confidencialidad del permiso, el sistema podría otorgar el

permiso automáticamente o el usuario del dispositivo podría tener que conceder la solicitud. Por ejemplo, si nuestra aplicación solicita permiso para activar la linterna del dispositivo, el sistema otorga ese permiso automáticamente. Sin embargo, si nuestra aplicación necesita leer los contactos del usuario, el sistema le solicita aprobar dicho permiso. Según la versión de la plataforma, el usuario otorga el permiso cuando se instala la aplicación (en Android 5.1 y versiones anteriores) o mientras esta se ejecuta (en Android 6.0 y versiones posteriores).

Sin embargo, si accedes a la documentación oficial, descubrirás que **la necesidad de los permisos para el acceso a los distintos datos y acciones varía con las distintas versiones**, circunstancia que debes tener muy en cuenta a la hora de planificar los permisos de tu aplicación.

1.1.- Permisos necesarios para nuestra aplicación.

Cuando desarrollamos nuestra aplicación, debemos tener en cuenta los casos en que esta use funcionalidades que requieran un permiso. Normalmente, una aplicación necesitará permisos siempre que use información o recursos que no haya creado, o cuando realice acciones que afecten el comportamiento del dispositivo o de demás aplicaciones. Por ejemplo, si una aplicación necesita acceder a Internet, usar la cámara del dispositivo o activar o desactivar la conexión wifi, necesitará los permisos correspondientes.

Nuestra aplicación solo necesita permisos para acciones que realiza directamente. No necesita permisos si solicita a otra aplicación que realice la tarea o proporcione la información (esta acción se denomina **delegación** de permisos). Por ejemplo, si debe leer la lista de direcciones del usuario, necesitará el permiso READ_CONTACTS. Pero si usa un **intent** para solicitar información de la aplicación de Contactos del usuario (preinstalada ya en los dispositivos), no necesitará permisos, aunque sí los necesitará la aplicación de Contactos.

Cada aplicación de Android se ejecuta en una zona de pruebas (también conocida como sandbox) con acceso limitado. Si una aplicación necesita usar recursos o información externos a su propia zona de pruebas, debe solicitar el permiso correspondiente haciendo una **declaración del permiso**.

La declaración de estos permisos hace que cada vez que tu aplicación requiera acceder a los recursos protegidos, esta debe pararse y solicitar al usuario el acceso. Esto puede provocar que tus posibles usuarios denieguen el acceso e incluso desinstalen la aplicación porque no comprendan la necesidad de su uso o porque no quieran otorgarlos. Debes saber que existen algunas alternativas que no requieren declarar estos permisos y debes evaluar si tu aplicación debe, por tanto, declararlos.

Además, es posible que existan otras aplicaciones en el terminal que ya cuenten con permisos para realizar la tarea necesaria; Android permite delegar ciertas acciones en otras aplicaciones que ya tengan concedido el acceso a los datos o las acciones requeridas.

Por ejemplo no requieren declarar permisos las siguientes acciones realizadas de esta forma:

- Para mostrar lugares cercanos solicita un código postal en lugar de acceder a la ubicación.
- Para hacer fotos o grabar vídeos usa las aplicaciones preinstaladas en el sistema (delega).
- Desde Android 11 (API) no es necesario solicitar permiso para acceder a contenido multimedia creado desde la propia aplicación previamente.
- Desde Android 10 no es necesario solicitar permisos para acceder a documentos.

Puedes ver más ejemplos en este enlace
[Evaluación de permisos en developer.android.com](#) 

1.2.- Agregar permisos al manifiesto.

Para declarar que nuestra aplicación necesita un permiso, dispondremos un elemento **<uses-permission>** en el fichero **AndroidManifest.xml** como elemento secundario del elemento de nivel superior **<manifest>**.

Ejemplo

En el manifiesto de una aplicación que necesite enviar mensajes SMS debería incluirse esta línea:

```
1 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
2     <uses-permission android:name="android.permission.SEND_SMS"/>
3
4     <application ...>
5         ...
6     </application>
</manifest>
```

El comportamiento del sistema después de declarar un permiso depende del grado de confidencialidad de este último. Si el permiso no afecta la privacidad del usuario, el sistema otorga el permiso automáticamente. Si el permiso puede otorgar acceso a información confidencial del usuario, el sistema solicita al usuario que apruebe la solicitud. En el apartado siguiente veremos cuáles son los diferentes tipos de permisos.

1.3.- Tipos de permisos del sistema.

Android categoriza los permisos en diferentes tipos, incluidos los del momento de la instalación, los permisos de tiempo de ejecución y los permisos especiales. Cada tipo de permiso indica el alcance de los datos restringidos a los que tu aplicación puede acceder, así como el alcance de las acciones restringidas que puede realizar, cuando el sistema le otorga ese permiso.

Permisos en el momento de la instalación

Los permisos en el momento de la instalación otorgan acceso limitado a los datos restringidos y permiten que realice acciones restringidas **que casi no afectan al sistema o a otras aplicaciones**. Cuando declaras permisos en el momento de la instalación en la aplicación, el sistema le otorga automáticamente los permisos cuando el usuario la instala. La tienda PlayStore muestra a los usuarios un aviso de permiso en el momento de la instalación al cuando ve la página de detalles de la aplicación. Hay dos tipos:

- **Permisos normales:** Permiten el acceso a los datos y las acciones que se extienden más allá de la zona de pruebas de tu aplicación. Sin embargo, los datos y las acciones presentan muy poco riesgo para la privacidad del usuario y el funcionamiento de otras aplicaciones. Puedes buscar aquellos permisos normales en

[esta página de referencia de Android sobre Permisos en el Manifest](#)  localizando aquellos que tengan indicado **Protection level: normal**.

- **Permisos de firma:** Permiten el acceso a certificados digitales instalados para firmar digitalmente o identificar a una persona o entidad a través de su cuenta para descargar aplicaciones o acceder a servicios. Puedes buscar aquellos permisos de firma en

[esta página de referencia de Android sobre Permisos en el Manifest](#)  localizando aquellos que tengan indicado **Protection level: signature**.

Permisos de tiempo de ejecución (o peligrosos)

Estos permisos abarcan áreas en las cuales la aplicación requiere datos o recursos que incluyen información privada del usuario, o bien que podrían afectar los datos almacenados del usuario o el funcionamiento de otras aplicaciones. Cuando la aplicación solicita un permiso en tiempo de ejecución, el sistema presenta un mensaje de este permiso que debe ser aceptado por el usuario.

Muchos permisos en tiempo de ejecución acceden a los **datos privados del usuario**, un tipo especial de datos restringidos que incluye información que puede ser sensible. Algunos ejemplos de datos privados del usuario incluyen la ubicación y la información de contacto del usuario, la lista de los contactos del usuario, etc.

El micrófono, la cámara y la ubicación brindan acceso a información **particularmente sensible**. Por lo tanto, debes evitar acceder desde tu aplicación a estos dispositivos sin haber obtenido el permiso explícito para ello (permisos CAMERA para la cámara, RECORD_AUDIO para el micro y ACCESS_COARSE_LOCATION o ACCESS_FINE_LOCATION para la localización).

Puedes buscar aquellos permisos peligrosos en

esta [página de referencia de Android sobre Permisos en el Manifest](#)  localizando aquellos que tengan indicado **Protection level: dangerous**

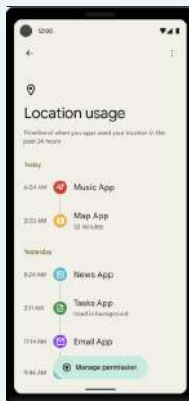
Permisos especiales

Estos permisos son particularmente confidenciales, por ello, la mayoría de las aplicaciones no deben usarlos. Solo la plataforma (el propio Android) y los OEM (fabricantes de los dispositivos como Samsung, Xiaomi, etc.) pueden definir permisos especiales. Además, la plataforma y los OEM suelen definir permisos especiales cuando quieren proteger el acceso a acciones particularmente importantes, como escribir sobre otras aplicaciones.

Si una aplicación necesita uno de estos permisos, debe declararlo en el manifiesto y enviar una intent en la que solicite la autorización del usuario. El sistema responde a la intent mostrando una pantalla de administración detallada al usuario. Puedes buscar aquellos permisos especiales en [esta página de referencia de Android sobre Permisos en el Manifest](#)  localizando aquellos que tengan indicado **Protection level: appop**



Debes conocer el Panel de Privacidad



(PMDM09_CONT_R4_Privacy-
dashboard.png)

 [Google Developers](https://developer.android.com/codelabs/android-navigation?hl=es-419#2)
([https://
developer.android.com/
codelabs/android-
navigation?hl=es-419#2](https://developer.android.com/codelabs/android-navigation?hl=es-419#2))
(Uso educativo nc)

En dispositivos compatibles que ejecutan Android 12 o versiones posteriores, aparece la pantalla del panel de privacidad en la configuración del sistema. En esta pantalla, los usuarios pueden acceder a distintas pantallas que se muestran cuando las aplicaciones acceden a la información de la ubicación, la cámara y el micrófono. En cada una, se muestra un cronograma del momento en que diferentes aplicaciones accedieron a un tipo específico de datos.

Además desde este panel puedes configurar los permisos a todas las aplicaciones, otorgando, quitando, obligando a preguntar, usando en segundo plano, etc.

1.4.- Grupos de permisos.

Todos los permisos peligrosos del sistema Android pertenecen a grupos de permisos. Si el dispositivo tiene Android 6.0 (nivel de API 23) instalado o superior y el atributo `targetSdkVersion` de la aplicación es 23 o un valor superior, se producirá el siguiente comportamiento del sistema cuando la aplicación solicita un permiso peligroso:

- Si una aplicación solicita un permiso peligroso incluido en su manifiesto y no tiene permisos actualmente en el grupo de permisos, el sistema muestra un cuadro de diálogo al usuario en el que se describe el grupo de permisos al cual la aplicación desea acceder. En el cuadro de diálogo no se describe el permiso específico dentro de ese grupo. Por ejemplo, si una aplicación solicita el permiso `READ_CONTACTS`, en el cuadro de diálogo del sistema se indica únicamente que la aplicación necesita acceso a los contactos del dispositivo. Si el usuario brinda la aprobación, el sistema otorga a la aplicación solamente el permiso que solicitó.
- Si una aplicación solicita un permiso peligroso incluido en su manifiesto y ya tiene otro permiso peligroso en el mismo grupo de permisos, el sistema lo otorga de inmediato sin interacción con el usuario. Por ejemplo, si a una aplicación ya se le otorgó el permiso `READ_CONTACTS` y luego esta solicita el permiso `WRITE_CONTACTS`, el sistema lo otorga de inmediato.

Cualquier permiso puede pertenecer a un grupo de permisos; entre ellos, los normales y los que define tu aplicación. Sin embargo, el grupo de un permiso solo afecta la experiencia del usuario si es peligroso. Puedes ignorar el grupo de permisos para los permisos normales.

Si el dispositivo tiene instalado Android 5.1 (nivel de API 22) o versiones anteriores, o si el atributo `targetSdkVersion` de la aplicación es 22 o inferior, el sistema solicita al usuario que otorgue el permiso en el momento de la instalación. Nuevamente, el sistema solo dice al usuario qué grupos de permisos necesita la aplicación y no los permisos individuales.

Algunos de estos grupos son: `CAMERA`, `CONTACTS`, `LOCATIONS`, `MICROPHONE`, `PHONE`, `NOTIFICATIONS`, `SMS`, `STORAGE`.

Puedes encontrar en este enlace los grupos de permisos de Android: [Página de referencia de Android sobre Grupos de Permisos en el Manifest](#) 

1-5.- Prácticas recomendadas en la gestión de los permisos.

Objetivos de privacidad

Los permisos de la aplicación se basan en las funciones de seguridad del sistema y ayudan a Android a cumplir los siguientes objetivos de la privacidad del usuario:

- **Control:** El usuario controla los datos que comparte con las aplicaciones.
- **Transparencia:** El usuario comprende los datos que usa una aplicación y el motivo por el que accede a estos datos.
- **Minimización de datos:** Una aplicación accede y usa solo los datos necesarios para una tarea o acción específica que el usuario invoca.

Prácticas recomendadas

Developers.android.com recomienda las siguientes prácticas:

- **Solicita una cantidad mínima de permisos:** Cuando el usuario solicita una acción específica en tu aplicación, esta solo debe solicitar los permisos que necesita para completarla. Según cómo uses los permisos, puede haber una forma alternativa para cumplir con el caso de uso de tu aplicación sin depender del acceso a información sensible.
- **Asocia permisos de tiempo de ejecución con acciones específicas:** Solicita permisos lo más tarde posible en los flujos de casos de uso de tu aplicación. Por ejemplo, si tu aplicación permite que los usuarios envíen mensajes de audio a otras personas, espera hasta que el usuario navegue a la pantalla de mensajes y presione el botón para enviar un mensaje de audio. Después de que el usuario presione el botón, la aplicación podrá solicitar acceso al micrófono.
- **Considera las dependencias de tu aplicación:** Cuando incluyes una biblioteca, también heredas sus requisitos de permisos. Ten en cuenta los permisos que requiere cada dependencia y para qué se usan.
- **Sé transparente:** Cuando solicitas permisos, sé claro con respecto a la información a la que accedes y a la razón por la que lo haces, de manera que los usuarios puedan tomar decisiones fundamentadas.
- **Crea accesos explícitos al sistema:** Cuando accedes a datos sensibles o al hardware, como la cámara o el micrófono, proporciona una indicación continua en tu aplicación si el sistema todavía no brinda estos indicadores. Este recordatorio ayuda a los usuarios a comprender exactamente cuándo tu aplicación accede a datos restringidos o realiza acciones restringidas.



Autoevaluación

A partir de Android 6.0, API 23 (MarshMallow)...

- ☐ El sistema, tras declarar un permiso en el archivo de manifiesto, siempre solicita al usuario que apruebe la solicitud de dicho permiso.
- ☐ El sistema solicita al usuario que apruebe la solicitud de un permiso, sin declarar dicho permiso en el archivo de manifiesto.
- ☐ El sistema, tras declarar un permiso en el archivo de manifiesto, ya no solicita al usuario que apruebe la solicitud de dicho permiso.
- ☐ El sistema, tras declarar un permiso en el archivo de manifiesto, puede solicitar o no, al usuario, que apruebe la solicitud de dicho permiso, dependiendo de su grado de confidencialidad.

Incorrecto

Incorrecto

Incorrecto

Opción correcta

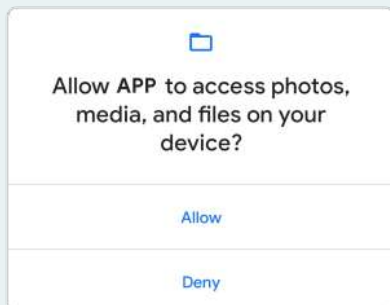
Solución

1. Incorrecto (#answer-424_144)
2. Incorrecto (#answer-424_10356)
3. Incorrecto (#answer-424_10359)
4. Opción correcta (#answer-424_10362)

2.- Solicitud de permisos en tiempo de ejecución.



Caso práctico



Q Google Developers (<https://developer.android.com/guide/platform?hl=es-419>) (Uso educativo nc)

¿Qué pueden ocurrir?

- En caso de que no conceda el permiso de forma explícita, la aplicación no deberá acceder a la cámara -responde Juan-. Si se lo concede, la aplicación tendrá acceso a la cámara.

- Nos ha quedado claro cómo opera Android con los permisos -le indica Juan a Ada-, ahora debemos ser capaces de construir una aplicación que acceda a la cámara del dispositivo para conseguir capturar una imagen.

- Me parece perfecto -responde Ada-. Para ello el primer paso es, como vimos previamente, declarar la necesidad de este permiso en el archivo de manifiesto de la aplicación Android, y el segundo paso es solicitar el permiso necesario al usuario.

A partir de Android 6.0 (nivel de API 23), los usuarios conceden permisos a las aplicaciones mientras se ejecutan, no cuando instalan la aplicación. Este enfoque simplifica el proceso de instalación de la aplicación, ya que el usuario no necesita conceder permisos cuando instala o actualiza la aplicación. También brinda al usuario mayor control sobre la funcionalidad de la aplicación; por ejemplo, un usuario podría optar por proporcionar a una aplicación de cámara acceso a esta, pero no a la ubicación del dispositivo. El usuario puede revocar los permisos en cualquier momento desde la pantalla de configuración de la aplicación o desde el nuevo Panel de Permisos (disponible desde Android 12).

Cómo hemos visto anteriormente existen distintos tipos de permisos en Android: permisos normales y de firma, peligrosos y especiales.

Los permisos normales y de firma no ponen en riesgo la privacidad del usuario de forma directa. Si tu aplicación tiene un permiso normal en su manifiesto, el sistema concede el permiso automáticamente.

Los permisos peligrosos pueden permitir que la aplicación acceda a información confidencial del usuario. Si tienes un permiso peligroso en el manifiesto, el usuario debe autorizar explícitamente a tu aplicación.

En todas las versiones de Android, nuestra aplicación debe declarar los permisos normales y peligrosos que necesita en su manifiesto, el efecto de esa declaración es diferente según la versión de sistema y el nivel de SDK de destino de nuestra aplicación:

- Si el dispositivo tiene Android 5.1 o una versión anterior, o el nivel de SDK de destino de tu aplicación es el 22 o uno inferior: Si tienes un permiso

peligroso en tu manifiesto, el usuario debe conceder el permiso cuando instale la aplicación; si no otorga el permiso, el sistema no instalará la aplicación.

- Si el dispositivo tiene Android 6.0 o una versión posterior, y el nivel de SDK de destino de tu aplicación es el 23 o uno posterior: Los permisos deben estar indicados en el manifiesto de la aplicación, y esta debe solicitar cada permiso peligroso que necesite mientras la aplicación esté en ejecución. El usuario puede conceder o negar cada permiso y la aplicación puede continuar ejecutándose con capacidades limitadas aun cuando el usuario rechace una solicitud de permiso.

Desde esta API 23 los usuarios pueden revocar permisos desde cualquier aplicación en cualquier momento, aunque la aplicación esté orientada a un nivel de API inferior. **Debemos probar nuestra aplicación para verificar que se comporte correctamente** cuando no cuente con un permiso necesario, independientemente del nivel de API al que esté orientada nuestra aplicación.

2.1.- Comprobación de existencia de permisos.

Si nuestra aplicación necesita un permiso peligroso, debemos verificar si tenemos ese permiso cada vez que realicemos una operación que lo requiera. El usuario siempre puede revocar el permiso. Por lo tanto, aunque la aplicación haya usado la cámara el día anterior, no podemos dar por hecho que seguirá teniendo ese permiso para el día en curso.

Para comprobar si tenemos un permiso, llamaremos al método **ContextCompat.checkSelfPermission()**.

Ejemplo

En este fragmento se muestra la manera de comprobar si la actividad tiene permiso para realizar operaciones de escritura en el calendario:

```
1 | // Assume thisActivity is the current activity
2 | int permissionCheck = ContextCompat.checkSelfPermission(thisActivity,
```

Si la aplicación tiene el permiso, el método muestra **PackageManager.PERMISSION_GRANTED** y esta puede continuar con la operación. Si la aplicación no tiene el permiso, el método muestra **PERMISSION_DENIED** y la aplicación debe solicitar permiso al usuario de manera explícita.

2.2.- Solicitar permisos.

Si nuestra aplicación necesita un permiso peligroso indicado en el manifiesto, debe solicitar al usuario que lo otorgue. Android ofrece varios métodos que podemos usar para solicitar un permiso. Cuando llamamos a estos métodos, aparece un diálogo de Android estándar que no se puede personalizar.

Si nuestra aplicación todavía no tiene el permiso que necesita, debemos llamar a uno de los métodos **requestPermissions()** para solicitar los permisos correspondientes. Tu aplicación pasa los permisos que necesita y también un *código de solicitud* de entero que tú especificas para identificar esta solicitud de permiso. Este método funciona de manera asincrónica: realiza la devolución inmediatamente y, cuando el usuario responde al cuadro de diálogo, el sistema llama al método *callback* de la aplicación con los resultados y pasa el mismo código de solicitud que la aplicación le pasó a **requestPermissions()**.

Ejemplo

El siguiente código verifica si la aplicación tiene permiso para leer los contactos del usuario y solicita el permiso si es necesario:

```
1 // Here, thisActivity is the current activity
2 if (ContextCompat.checkSelfPermission(thisActivity, Manifest.permission.READ_CONTACTS)
3     != PackageManager.PERMISSION_GRANTED) {
4     // Should we show an explanation?
5     if (ActivityCompat.shouldShowRequestPermissionRationale(thisActivity,
6         Manifest.permission.READ_CONTACTS)) {
7         // Show an explanation to the user *asynchronously* -- don't block
8         // this thread waiting for the user's response! After the user sees
9         // the explanation, try again to request the permission.
10    } else {
11        // No explanation needed, we can request the permission.
12        ActivityCompat.requestPermissions(thisActivity,
13            new String[]{Manifest.permission.READ_CONTACTS},
14            MY_PERMISSIONS_REQUEST_READ_CONTACTS);
15        // MY_PERMISSIONS_REQUEST_READ_CONTACTS is an
16        // app-defined int constant. The callback method gets the
17        // result of the request.
18    }
19 }
```

Cuando nuestra aplicación solicita permisos, el sistema muestra al usuario un cuadro de diálogo. Cuando el usuario responde, el sistema invoca el método **onRequestPermissionsResult()** de nuestra aplicación y le pasa la respuesta del usuario. Nuestra aplicación debe anular ese método para averiguar si se otorgó el permiso. El callback recibe el mismo código de solicitud que le pasaste a **requestPermissions()**.

Ejemplo

Si una aplicación solicita acceso a **READ_CONTACTS**, es posible que tenga el siguiente método callback:

```
1  @Override
2  public void onRequestPermissionsResult(int requestCode, String perm
3
4      switch (requestCode) {
5          case MY_PERMISSIONS_REQUEST_READ_CONTACTS: {
6              // If request is cancelled, the result arrays are empty
7              if (grantResults.length > 0
8                  && grantResults[0] == PackageManager.PERMISSION_GRANTED
9                  // permission was granted, yay! Do the
10                 // contacts-related task you need to do.
11             } else {
12                 // permission denied, boo! Disable the
13                 // functionality that depends on this permission.
14             }
15             return;
16         }
17         // other 'case' lines to check for other
18         // permissions this app might request
19     }
}
```

El cuadro de diálogo que muestra el sistema describe el grupo de permisos al que nuestra aplicación necesita acceder, pero no indica el permiso específico. Por ejemplo, si solicitamos el permiso **READ_CONTACTS**, el cuadro de diálogo del sistema simplemente indica que nuestra aplicación necesita acceder a los contactos del dispositivo. El usuario solamente debe otorgar el permiso una vez para cada grupo de permisos. Si nuestra aplicación requiere otros permisos de ese grupo (que se indican en el manifiesto de tu aplicación), el sistema los concede automáticamente. Cuando solicitamos el permiso, el sistema llama a tu

método callback **onRequestPermissionsResult()** y pasa **PERMISSION_GRANTED**, de la misma manera en que lo haría si el usuario hubiera aceptado explícitamente nuestra solicitud a través del cuadro de diálogo del sistema.

Por ejemplo, supón que indicamos **READ_CONTACTS** y **WRITE_CONTACTS** en el manifiesto de nuestra aplicación. Si solicitamos **READ_CONTACTS**, el usuario concede el permiso y luego solicitamos **WRITE_CONTACTS**, el sistema nos otorga de inmediato ese permiso sin interactuar con el usuario.

Si un usuario rechaza una solicitud de permiso, nuestra aplicación debe tomar una medida adecuada. Por ejemplo, nuestra aplicación podría mostrar un diálogo en el que se explique por qué no podría realizar la acción solicitada por el usuario para la cual se requiere ese permiso.

Cuando el sistema solicita al usuario que otorgue un permiso, el usuario tiene la opción de indicar al sistema que no solicite ese permiso de nuevo. En ese caso, cuando la aplicación use **requestPermissions()** para solicitar ese permiso nuevamente, el sistema rechazará la solicitud de inmediato. El sistema llama a nuestro método callback **PERMISSION_DENIED** y pasa **onRequestPermissionsResult()**, de la misma manera en que lo haría si el usuario hubiera rechazado explícitamente nuestra solicitud una vez más. Esto significa que cuando llamamos a **requestPermissions()**, no podemos suponer que haya existido interacción directa con el usuario.

2.3.- Ejercicio resuelto.



Ejercicio Resuelto



(p903-img001.png)

[Google Developers](https://developer.android.com/studio)
(https://
developer.android.com/
studio)_(Uso educativo nc)

Crear una aplicación que permita realizar una llamada telefónica a un número de teléfono que podrá indicarse a través de un campo de texto.

La aplicación debe implementar un correcto uso del sistema de permisos de Android, de forma que la primera vez que se ejecute la aplicación, debe solicitar al usuario permiso para poder llamar por teléfono ("**android.permission.CALL_PHONE**"), tal y como se muestra en la captura de pantalla inicial.

Una vez autorizado, se cargará una interfaz similar a la mostrada en la figura 1 (busca en el sistema o en Internet una imagen apropiada para representar el botón de llamada). Si ahora escribimos un número de teléfono se mostrará una interfaz similar a la mostrada en la figura 2. Simplemente pulsaremos el botón de llamada y ya procederá a realizarse la llamada, tal como se muestra en la figura 3.

Figura 1. Tras conceder los permisos.



(p903-img002.1.png)

[Google Developers](https://developer.android.com/studio)
(https://
developer.android.com/
studio)_(Uso educativo nc)

Figura 2. Escribir un nº de tfno.



(p903-img003.1.png)

 [Google Developers](https://developer.android.com/studio)

([https://](https://developer.android.com/studio)
[developer.android.com/](https://developer.android.com/studio)
[studio](https://developer.android.com/studio)).(Uso educativo nc)

Figura 3. Llamada realizada.



(p903-img004.1.png)

 [Google Developers](https://developer.android.com/studio)

([https://](https://developer.android.com/studio)
[developer.android.com/](https://developer.android.com/studio)
[studio](https://developer.android.com/studio)).(Uso educativo nc)

Mostrar retroalimentación

AndroidManifest.xml (#exe-accordion-4-0)

Debemos añadir esta línea al fichero:

```
1 | <uses-permission android:name="android.permission.C
```

activity_main.xml
(#exe-accordion-5-0)

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <RelativeLayout xmlns:android="http://schemas.android.com/apk/res-a
3
4      xmlns:app="http://schemas.android.com/apk/res-a
5      xmlns:tools="http://schemas.android.com/tools"
6
7      android:layout_width="match_parent"
8      android:layout_height="match_parent"
9      tools:context="com.cidead.pmdm.tarea92llamada.M
10
11
12      <EditText
13
14          android:id="@+id/etNumeroTelefono"
15          android:layout_width="match_parent"
16          android:layout_height="wrap_content"
17          android:hint="marque teléfono"
18          android:layout_marginTop="20dp" />
19
20
21
22      <Button
23
24          android:id="@+id/btnLlamar"
25          android:layout_width="100dp"
26          android:layout_height="100dp"
27          android:layout_below="@id/etNumeroTelefono"
28          android:layout_marginTop="20dp"
29          android:background="@drawable/telefono"
30
31          android:layout_centerHorizontal="true" />
32
33  </RelativeLayout>

```

MainActivity.java
 (#exe-accordion-6-0)

```
1 package com.cidead.pndm.tarea92llamada;
2
3
4 import androidx.annotation.NonNull;
5 import androidx.appcompat.app.AppCompatActivity;
6
7 import androidx.core.app.ActivityCompat;
8 import androidx.core.content.ContextCompat;
9
10
11 import android.Manifest;
12 import android.content.Intent;
13 import android.content.pm.PackageManager;
14 import android.net.Uri;
15 import android.os.Bundle;
16 import android.text.TextUtils;
17 import android.view.View;
18 import android.widget.Button;
19 import android.widget.EditText;
20 import android.widget.Toast;
21
22
23
24
25
26
27 public class MainActivity extends AppCompatActivity
28
29
30     private static final int MAKE_CALL_PERMISSION_R
31
32
33     private EditText etNumeroTelefono;
34     private Button btnLlamar;
35
36
37     @Override
38
39     protected void onCreate(Bundle savedInstanceState)
40         super.onCreate(savedInstanceState);
41         setContentView(R.layout.activity_main);
42
43
44         btnLlamar = (Button) findViewById(R.id.btnL
45         etNumeroTelefono = (EditText) findViewById(
46
47
48
49         btnLlamar.setOnClickListener(new View.OnCli
50
51         @Override
52         public void onClick(View view) {
53
```



```

        String numeroTelefono = etNumeroTelefono.getText().toString();

        if (!TextUtils.isEmpty(numeroTelefono)) {
            if (comprobarPermiso(Manifest.permission.CALL_PHONE)) {
                String dial = "tel:" + numeroTelefono;
                if (ActivityCompat.checkSelfPermission(this, Manifest.permission.CALL_PHONE) != PackageManager.PERMISSION_GRANTED) {
                    // TODO: Consider calling
                    //    ActivityCompat#requestPermissions
                    // here to request the missing permissions, and
                    //    ActivityCompat#shouldShowRequestPermissionRationale
                    // to handle the case where the user has denied the
                    // permission.
                    return;
                }
                startActivity(new Intent(Intent.ACTION_DIAL, Uri.parse(dial)));
            } else {
                Toast.makeText(MainActivity.this, "No se puede llamar", Toast.LENGTH_SHORT).show();
            }
        } else {
            Toast.makeText(MainActivity.this, "El número de teléfono no puede estar vacío", Toast.LENGTH_SHORT).show();
        }
    });

    if (comprobarPermiso(Manifest.permission.CALL_PHONE)) {
        btnLlamar.setEnabled(true);
    } else {
        btnLlamar.setEnabled(false);
        ActivityCompat.requestPermissions(this, new String[]{Manifest.permission.CALL_PHONE}, 1);
    }
}

private boolean comprobarPermiso(String permission) {
    return ActivityCompat.checkSelfPermission(this, permission) == PackageManager.PERMISSION_GRANTED;
}

```

```
@Override
public void onRequestPermissionsResult(int requestCode,
    String[] permissions, int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    switch (requestCode) {
        case MAKE_CALL_PERMISSION_REQUEST_CODE:
            if (grantResults.length > 0 && (grantResults[0] == PackageManager.PERMISSION_GRANTED)) {
                btnLlamar.setEnabled(true);
                Toast.makeText(this, "Para llamar", Toast.LENGTH_SHORT).show();
            }
            return;
        default:
            return;
    }
}
```

II - Sensores.



Caso práctico



Q [Google Developers \(https://developer.android.com/guide/topics/sensors/gnss?hl=es-419\)](https://developer.android.com/guide/topics/sensors/gnss?hl=es-419). (Uso educativo nc)

De nuevo, el equipo se enfrenta a un nuevo reto a partir de las necesidades concretas de un cliente de BK Programación que se dedica a la venta de bicicletas.

- Tenemos una petición de un cliente que desea desarrollar una aplicación Android para los compradores de sus bicicletas - informa Ada a su equipo, que espera expectante su nuevo reto-. El objetivo principal de la aplicación es enviar mensajes publicitarios, cupones de descuento y otras ofertas para fidelizar a

sus clientes.

- Pero -señala María, siempre muy atenta a las explicaciones de Ada-, para que la aplicación sea instalada por los usuarios, esta debe ofrecer algún servicio.

- Buena puntualización -continúa explicando Ada-. En este caso se desea ofrecer un conjunto de valores relacionados con una actividad deportiva, mostrando la velocidad a la que se desplazan los ciclistas, la distancia recorrida, la temperatura actual, etc. y otros parámetros calculados como las calorías quemadas durante el ejercicio, la velocidad media durante la actividad, etc.

- Vamos a investigar qué tipo de sensores tenemos a nuestro alcance y cómo manejarlos -anima Pedro al equipo.

El equipo descubre que los dispositivos Android incluyen un conjunto importante de sensores bastante precisos capaces de capturar información muy interesante para desarrollar este tipo de aplicaciones. Aprender a manejarlos es el reto que tienen por delante para ofrecer a los usuarios datos calculados a partir de la información real capturada desde el dispositivo.



Q [Ministerio de Educación y Formación Profesional.](https://www.minedu.es/)

(Dominio público)

de Educación y Formación Profesional.

 [Aviso Legal](#) 

1.- Tipos de sensores.

Existen sensores basados en hardware y sensores basados en software:

- Los **sensores basados en hardware** son componentes físicos integrados en el dispositivo. Derivan sus datos midiendo directamente las propiedades ambientales específicas, como la aceleración, la intensidad del campo geomagnético o el cambio angular.
- Los **sensores basados en software** no son dispositivos físicos, aunque imitan a los sensores basados en hardware. Los sensores basados en software derivan sus datos de uno o más de los sensores basados en hardware y, a veces, se los denomina sensores virtuales o sensores sintéticos. El sensor de aceleración lineal y el sensor de gravedad son ejemplos de sensores basados en software.



Q Stanley Ng (Pexels)

Pocos dispositivos con Android tienen todos los sensores especificados en la tabla que son los compatibles con Android. Por ejemplo, la mayoría de los dispositivos y tabletas tienen un acelerómetro y un magnetómetro, pero pocos dispositivos tienen barómetros o termómetros. Además, un dispositivo puede tener más de un sensor de un tipo determinado. Por ejemplo, un dispositivo puede tener dos sensores de gravedad, cada uno con un rango diferente.

Tipos de sensores compatibles con la plataforma Android

Sensor	Tipo	Descripción	Usos comunes
TYPE_ACCELEROMETER 	Hardware	Mide, en m/s^2 , la fuerza de aceleración que se aplica a un dispositivo en los tres ejes físicos (x, y, z), incluida la fuerza de gravedad.	Detección de movimiento (agitación, inclinación, etc.).
TYPE_AMBIENT_TEMPERATURE 	Hardware	Mide la temperatura ambiente de la habitación en grados Celsius ($^{\circ}\text{C}$). Consulta la siguiente nota.	Supervisión de la temperatura del aire.
TYPE_GRAVITY 	Software o hardware	Mide, en m/s^2 , la fuerza de gravedad que se aplica a un dispositivo en los tres ejes físicos (x, y, z).	Detección de movimiento (agitación, inclinación, etc.).

TYPE_GYROSCOPE 	Hardware	Mide, en rad/s , la velocidad de rotación de un dispositivo alrededor de cada uno de los tres ejes físicos (x, y, z).	Detección de rotación (agitación, giro, etc.).
TYPE_LIGHT 	Hardware	Mide el nivel de luz ambiental (iluminación) en lx.	Control del brillo de la pantalla.
TYPE_LINEAR_ACCELERATION 	Software o hardware	Mide, en m/s^2 , la fuerza de aceleración que se aplica a un dispositivo en los tres ejes físicos (x, y, z), excluyendo la fuerza de gravedad.	Supervisión de la aceleración a lo largo de un solo eje.
TYPE_MAGNETIC_FIELD 	Hardware	Mide el campo geomagnético ambiental de los tres ejes físicos (x, y, z) en μT .	Creación de una brújula.
TYPE_ORIENTATION 	Software	Mide los grados de rotación de un dispositivo alrededor de los tres ejes físicos (x, y, z). A partir de la API nivel 3, puedes obtener las matrices de inclinación y de rotación mediante el uso de los sensores de gravedad y de campo geomagnético del dispositivo, usando el sensor de gravedad junto con el método <code>getRotationMatrix()</code>  .	Determinación de la posición del dispositivo.
TYPE_PRESSURE 	Hardware	Mide la presión del aire del ambiente en hPa o mbar.	Supervisa los cambios de la presión del aire.
TYPE_PROXIMITY 	Hardware	Mide, en cm, la proximidad de un objeto con respecto a la pantalla de visualización de un dispositivo. Este sensor en general se usa para determinar si un dispositivo manual se	Posición del teléfono durante una llamada.

		está sosteniendo cerca del oído de una persona.	
TYPE_RELATIVE_HUMIDITY 	Hardware	Mide en valor de porcentaje (%) la humedad relativa del ambiente.	Supervisa el punto de condensación la humedad absoluta y la humedad relativa.
TYPE_ROTATION_VECTOR 	Software o hardware	Mide la orientación de un dispositivo mediante los tres elementos del vector de rotación del dispositivo.	Detección de movimiento ; detección de rotación.
TYPE_TEMPERATURE 	Hardware	Mide la temperatura del dispositivo en grados Celsius (°C). La implementación de este sensor varía según el dispositivo; en la API nivel 14 se reemplazó por el sensor TYPE_AMBIENT_TEMPERATURE  .	Supervisión de temperatura.

2.- Framework o entorno de trabajo del sensor.

Podemos acceder a estos sensores y adquirir datos sin procesar del sensor utilizando el *framework* de sensores de Android. El *framework* del sensor es parte del paquete **android.hardware** e incluye las siguientes clases e interfaces:

- **SensorManager:** Podemos usar esta clase para crear una instancia del servicio del sensor. Esta clase proporciona varios métodos para acceder y enumerar sensores, registrar y anular el registro de receptores de eventos de sensores y adquirir información de orientación. Esta clase también proporciona varias constantes de sensor que se utilizan para informar la precisión del sensor, establecer velocidades de adquisición de datos y calibrar sensores.
- **Sensor:** Podemos usar esta clase para crear una instancia de un sensor específico. Esta clase proporciona varios métodos que nos permiten determinar las capacidades de un sensor.
- **SensorEvent:** El sistema usa esta clase para crear un objeto de evento sensor, que proporciona información sobre un evento sensor. Un objeto de evento de sensor incluye la siguiente información:
 - los datos del sensor en bruto,
 - el tipo de sensor que generó el evento,
 - la precisión de los datos
 - y la marca de tiempo del evento.
- **SensorEventListener:** Podemos usar esta interfaz para crear dos métodos de devolución de llamada que reciben notificaciones (eventos del sensor) cuando cambian los valores del sensor o cuando cambia la precisión del sensor.

En una aplicación típica, utilizaremos estas API relacionadas con el sensor para realizar dos tareas básicas:

- **Identificación de sensores y capacidades del sensor**

La identificación de sensores y capacidades del sensor en el tiempo de ejecución es útil si su aplicación tiene características que dependen de capacidades o tipos de sensores específicos. Por ejemplo, es posible que desee identificar todos los sensores que están presentes en un dispositivo y desactivar las funciones de la aplicación que dependen de sensores que no están presentes. Del mismo modo, es posible que desee identificar todos los sensores de un tipo determinado para que pueda elegir la implementación del sensor que tenga el rendimiento óptimo para su aplicación.

- **Control de los eventos del sensor**

La monitorización de los eventos del sensor es la forma de adquirir los datos del sensor sin procesar. Se produce un evento de sensor cada vez que un sensor detecta un cambio en los parámetros que está midiendo. Un evento de sensor le proporciona cuatro elementos de información: el nombre del sensor que activó el evento, la marca de tiempo del evento, la precisión del evento y los datos brutos del sensor que

desencadenaron el evento.

3.- Disponibilidad del sensor.

Si bien la disponibilidad del sensor varía de un dispositivo a otro, también puede variar entre las versiones de Android. Esto se debe a que los sensores de Android se han introducido a lo largo de varias versiones de la plataforma. Por ejemplo, se introdujeron muchos sensores en Android 1.5 (nivel 3 de la API), pero algunos no se implementaron y no estuvieron disponibles para su uso hasta Android 2.3 (API de nivel 9).

Asimismo, se introdujeron varios sensores en Android 2.3 (Nivel API 9) y Android 4.0 (Nivel API 14). Actualmente, prácticamente todos los dispositivos incluyen estos sensores, aunque puedes buscar aquellos permisos peligrosos en [esta página de referencia de Disponibilidad de los sensores](#) 

Dos sensores (**TYPE_ORIENTATION** y **TYPE_TEMPERATURE**) han quedado en desuso y reemplazados por sensores nuevos y mejores.

4.- Identificación de sensores.

El marco del sensor de Android proporciona varios métodos que le permiten determinar en tiempo de ejecución qué sensores están en un dispositivo. La API también proporciona métodos que le permiten determinar las capacidades de cada sensor, como su rango máximo, su resolución y sus requisitos de potencia.

Para identificar los sensores que están en un dispositivo, primero necesitamos obtener una referencia al servicio del sensor. Para hacerlo, crearemos una instancia de la clase **SensorManager** llamando al método **getSystemService()** y pasando el argumento **SENSOR_SERVICE**.

Ejemplo

```
1 | private SensorManager mSensorManager;  
2 |  
3 | ...  
   mSensorManager = (SensorManager) getSystemService (Context.SENSOR_
```

A continuación, podemos obtener una lista de cada sensor en un dispositivo llamando al método **getSensorList()** y utilizando la constante **TYPE_ALL**.

Ejemplo

```
1 | Lista <Sensor> deviceSensors = mSensorManager.getSensorList (Senso
```

Si deseamos enumerar todos los sensores de un tipo determinado, podemos usar otra constante en lugar de **TYPE_ALL** como **TYPE_GYROSCOPE**, **TYPE_LINEAR_ACCELERATION** o **TYPE_GRAVITY**.

También podemos determinar si existe un tipo específico de sensor en un dispositivo utilizando el método **getDefaultSensor()** y pasando la constante de tipo para un sensor específico. Si un dispositivo tiene más de un sensor de un tipo determinado, uno de los sensores debe designarse como el sensor predeterminado. Si no existe un sensor predeterminado para un tipo determinado de sensor, la llamada al método devuelve nulo, lo que significa que el dispositivo no tiene ese tipo de sensor.

Ejemplo

El siguiente código verifica si hay un magnetómetro en un dispositivo:

```
1  private SensorManager mSensorManager;  
2  ...  
3  
4  mSensorManager = (SensorManager) getSystemService (Context.SENSOR_...  
5  if (mSensorManager.getDefaultSensor (Sensor.TYPE_MAGNETIC_FIELD)!...  
6      // ¡Éxito! Hay un magnetómetro.  
7  }  
8  
9  else {  
    // ¡Fallo! Sin magnetómetro  
  }
```

Android no requiere que los fabricantes de dispositivos creen ningún tipo particular de sensores en sus dispositivos, por lo que los dispositivos pueden tener una amplia gama de configuraciones de sensores.

Otro método útil es el método **getMinDelay()**, que devuelve el intervalo de tiempo mínimo (en microsegundos) que un sensor puede usar para detectar los datos. Cualquier sensor que devuelve un valor distinto de cero para el método **getMinDelay()** es un sensor de transmisión. Los sensores de transmisión por secuencias detectan los datos a intervalos regulares y se introdujeron en Android 2.3 (nivel de API 9). Si un sensor devuelve cero cuando llama al método **getMinDelay()**, significa que el sensor no es un sensor de transmisión porque informa los datos solo cuando hay un cambio en los parámetros que está detectando.

El método **getMinDelay()** es útil porque nos permite determinar la velocidad máxima a la que un sensor puede adquirir datos. Si ciertas funciones en su aplicación requieren altas tasas de adquisición de datos o un sensor de transmisión, puede utilizar este método para determinar si un sensor cumple con esos requisitos y luego habilitar o deshabilitar, en consecuencia, las características relevantes en su aplicación.

Precaución: la velocidad máxima de adquisición de datos de un sensor no es necesariamente la velocidad a la que el marco del sensor entrega los datos del sensor a su aplicación. El marco del sensor informa los datos a través de los eventos del sensor, y varios factores influyen en la velocidad a la que su aplicación recibe los eventos del sensor.



Autoevaluación

Para obtener una lista de todos los sensores de un dispositivo utilizamos la constante:

- ☐ TYPE_GRAVITY
- ☐ TYPE_ALL
- ☐ TYPE_GYROSCOPE
- ☐ TYPE_LINEAR_ACCELERATION

No es correcto, vuelve a leer el apartado.

Correcto.

No es correcto, repasa el texto de nuevo

No es correcto, esta constante proporciona otra información.

Solución

1. Incorrecto (#answer-402_89)
2. Opción correcta (#answer-402_10603)
3. Incorrecto (#answer-402_10606)
4. Incorrecto (#answer-402_10609)

5.- Monitorización de eventos del sensor.

Para monitorizar datos de sensores necesitamos implementar dos métodos contemplados en la interfaz **SensorEventListener**, que serán **onAccuracyChanged():onAccuracyChanged()** y **onSensorChanged()**. El sistema Android llama a estos métodos siempre que ocurran ciertas circunstancias.

La precisión de un sensor cambia.

En este caso, el sistema invoca el método **onAccuracyChanged()**, que le proporciona una referencia al objeto **Sensor** que cambió y la nueva precisión del sensor. La precisión está representada por una de las cuatro constantes de estado: **SENSOR_STATUS_ACCURACY_LOW**, **SENSOR_STATUS_ACCURACY_MEDIUM**, **SENSOR_STATUS_ACCURACY_HIGH** o **SENSOR_STATUS_UNRELIABLE**.

Un sensor informa un nuevo valor.

En este caso, el sistema invoca el método **onSensorChanged()**, que le proporciona un objeto **SensorEvent**. Un objeto **SensorEvent** contiene información sobre los nuevos datos del sensor, que incluyen: la precisión de los datos, el sensor que generó los datos, la marca de tiempo en la que se generaron los datos y los nuevos datos que el sensor registró.

Ejemplo

El siguiente código muestra cómo usar el método **onSensorChanged()** para controlar los datos del sensor de luz. Este ejemplo muestra los datos del sensor en un *TextView* que está definido en el archivo **main.xml** como **sensor_data**.

```
1 public class SensorActivity extends Activity implements SensorEventListener {
2     private SensorManager mSensorManager;
3
4     private Sensor mLight;
5
6
7     @Override
8     public final void onCreate(Bundle savedInstanceState) {
9         super.onCreate(savedInstanceState);
10        setContentView(R.layout.main);
11
12        mSensorManager = (SensorManager) getSystemService(Context.SENSOR_SERVICE);
13
14        mLight = mSensorManager.getDefaultSensor(Sensor.TYPE_LIGHT);
15    }
16
17
18    @Override
19
20    public final void onAccuracyChanged(Sensor sensor, int accuracy) {
21        // Do something here if sensor accuracy changes.
22    }
23
24    public final void onSensorChanged(SensorEvent event) {
25        // Do something here if there is a change in raw sensor data.
26    }
27 }
```

```

    }

    @Override
    public final void onSensorChanged(SensorEvent event) {
        // The light sensor returns a single value.
        // Many sensors return 3 values, one for each axis.
        float lux = event.values[0];
        // Do something with this sensor value.
    }

    @Override
    protected void onResume() {
        super.onResume();
        mSensorManager.registerListener(this, mLight, SensorManager.SENSOR_DELAY_NORMAL);
    }

    @Override
    protected void onPause() {
        super.onPause();
        mSensorManager.unregisterListener(this);
    }
}

```

En el ejemplo, el retraso de datos predeterminado (**SENSOR_DELAY_NORMAL**) se especifica cuando se invoca el método **registerListener()**. El retardo de datos (o tasa de muestreo) controla el intervalo en el cual los eventos del sensor se envían a su aplicación a través del método de devolución de llamada **onSensorChanged()**. La demora de datos predeterminada es adecuada para monitorear cambios típicos de orientación de pantalla y utiliza un retraso de 200.000 microsegundos. Podemos especificar otros retrasos de datos, como **SENSOR_DELAY_GAME** (retardo de 20.000 microsegundos), **SENSOR_DELAY_UI** (retardo de 60.000 microsegundos) o **SENSOR_DELAY_FASTEST** (retardo de 0 microsegundos). A partir de Android 3.0 (API nivel 11) también podemos especificar el retraso como un valor absoluto (en microsegundos).

La demora que especifiquemos solo es un retraso sugerido. El sistema Android y otras aplicaciones pueden alterar este retraso. Como práctica recomendada, debemos especificar la demora más grande que podamos porque el sistema generalmente usa un retraso menor que el especificado por nosotros (es decir, debemos elegir la frecuencia de muestreo más lenta que aún satisfaga las necesidades de nuestra aplicación). Usar un retraso mayor impone una carga

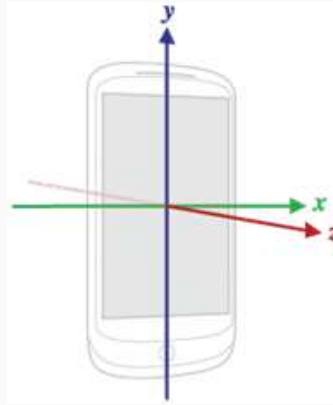
menor en el procesador y, por lo tanto, utilizará menos energía.

No hay un método público para determinar la velocidad a la que el framework del sensor está enviando eventos del sensor a nuestra aplicación; sin embargo, podemos usar las marcas de tiempo que están asociadas con cada evento del sensor para calcular la tasa de muestreo en varios eventos.

También es importante tener en cuenta que este ejemplo utiliza los métodos de devolución de llamada **onResume()** y **onPause()** para registrar y anular el registro del detector de eventos del sensor. Como práctica recomendada, siempre debemos desactivar los sensores que no necesitamos, especialmente cuando su actividad está en pausa. Si no lo hacemos, agotaremos la batería en unas pocas horas debido a que algunos sensores tienen requisitos de energía considerables y pueden agotar rápidamente la energía de la batería. El sistema no desactivará los sensores automáticamente cuando la pantalla se apaga.

6.- Sistema de coordenadas del sensor.

En general, el marco del sensor utiliza un sistema de coordenadas de tres ejes estándar para expresar valores de datos. Para la mayoría de los sensores, el sistema de coordenadas se define en relación con la pantalla del dispositivo cuando el dispositivo se mantiene en su orientación predeterminada (consulte la figura). Cuando un dispositivo se mantiene en su orientación predeterminada, el eje X es horizontal y apunta hacia la derecha, el eje Y es vertical y apunta hacia arriba, y el eje Z apunta hacia el exterior de la pantalla. En este sistema, las coordenadas detrás de la pantalla tienen valores Z negativos. Este sistema de coordenadas es utilizado por los siguientes sensores:



Q [Google Developers](#) (Uso educativo no)

- Sensor de aceleración
- Sensor de gravedad
- Giroscopio
- Sensor de aceleración lineal
- Sensor de campo geomagnético

El punto más importante para entender sobre este sistema de coordenadas es que los ejes no se intercambian cuando cambia la orientación de la pantalla del dispositivo; es decir, el sistema de coordenadas del sensor nunca cambia a medida que el dispositivo se mueve. Este comportamiento es el mismo que el del sistema de coordenadas OpenGL.

Otro punto que debemos entender es que nuestra aplicación no debe asumir que la orientación natural (por defecto) de un dispositivo es vertical. La orientación natural para muchos dispositivos de tableta es el paisaje. Y el sistema de coordenadas del sensor siempre se basa en la orientación natural de un dispositivo.

Finalmente, si nuestra aplicación relaciona los datos del sensor con la visualización en pantalla, debemos usar el método **getRotation()** para determinar la rotación de la pantalla y luego usar el método **remapCoordinateSystem()** para mapear las coordenadas del sensor a las coordenadas de la pantalla. Debemos hacer esto incluso si nuestro archivo **AndroidManifest.xml** especifica la visualización solo retrato.

7.- Sensor de gravedad.

El sensor de gravedad proporciona un vector tridimensional que indica la dirección y la magnitud de la gravedad. Normalmente, este sensor se usa para determinar la orientación relativa del dispositivo en el espacio.



Q [manuel m. v.](#) (CC BY-NC-ND)

El siguiente código muestra cómo obtener una instancia del sensor de gravedad predeterminado:

Código

```
1 private SensorManager mSensorManager;  
2 private Sensor mSensor;  
3  
4 ...  
5 mSensorManager = (SensorManager) getSystemService(Context.SENSOR_SE  
    mSensor = mSensorManager.getDefaultSensor(Sensor.TYPE_GRAVITY);
```

Las unidades son las mismas que las utilizadas por el sensor de aceleración m/s^2 , y el sistema de coordenadas es el mismo que el usado por el sensor de aceleración.

Cuando un dispositivo está en reposo, la salida del sensor de gravedad debe ser idéntica a la del acelerómetro.

8.- Sensor de aceleración o acelerómetro.

Un sensor de aceleración mide la aceleración aplicada al dispositivo, incluida la fuerza de la gravedad. El siguiente código muestra cómo obtener una instancia del sensor de aceleración predeterminado:

Código

```
1 private SensorManager mSensorManager;  
2     sensor privado mSensor;  
3  
4     ...  
5     mSensorManager = (SensorManager) getSystemService (Context.SENSOR_  
        mSensor = mSensorManager.getDefaultSensor (Sensor.TYPE_ACCELEROMET
```

Conceptualmente, un sensor de aceleración determina la aceleración que se aplica a un dispositivo (A_D) al medir las fuerzas que se aplican al sensor en sí (F_S) usando la siguiente relación:

$$A_D = \left(\frac{1}{\text{mass}} \right) \sum F_S$$

Sin embargo, la fuerza de la gravedad siempre influye en la aceleración medida de acuerdo con la siguiente relación:

$$A_D = -g - \left(\frac{1}{\text{mass}} \right) \sum F_S$$

Por esta razón, cuando el dispositivo está apoyado sobre una mesa (y no está acelerando), el acelerómetro muestra una magnitud de $(g = 9.81 \text{ m/s}^2)$. De manera similar, cuando el dispositivo está en caída libre y por lo tanto acelera rápidamente hacia el suelo a (9.81 m/s^2) , su acelerómetro lee una magnitud de $(g = 0 \text{ m/s}^2)$. Por lo tanto, para medir la aceleración real del dispositivo, la contribución de la fuerza de gravedad debe eliminarse de los datos del acelerómetro. Esto se puede lograr aplicando un filtro de paso alto. Por el contrario, se puede usar un filtro de paso bajo para aislar la fuerza de la gravedad.

Ejemplo

Uso de filtro de paso bajo para aislar la fuerza de la gravedad:

```
1 public void onSensorChanged(SensorEvent event){  
2  
3  
4     // In this example, alpha is calculated as t / (t + dT),  
5
```

```

// where t is the low-pass filter's time-constant and
// dT is the event delivery rate.

final float alpha = 0.8;

// Isolate the force of gravity with the low-pass filter.
gravity[0] = alpha * gravity[0] + (1 - alpha) * event.values[0];
gravity[1] = alpha * gravity[1] + (1 - alpha) * event.values[1];
gravity[2] = alpha * gravity[2] + (1 - alpha) * event.values[2];

// Remove the gravity contribution with the high-pass filter.
linear_acceleration[0] = event.values[0] - gravity[0];
linear_acceleration[1] = event.values[1] - gravity[1];
linear_acceleration[2] = event.values[2] - gravity[2];

}

```

Podemos usar muchas técnicas diferentes para filtrar los datos del sensor. El ejemplo de código anterior usa una constante de filtro simple (alfa) para crear un filtro de paso bajo. Esta constante de filtro se deriva de una constante de tiempo (t), que es una representación aproximada de la latencia que el filtro agrega a los eventos del sensor, y la tasa de entrega de eventos del sensor (dt). El ejemplo de código usa un valor alfa de 0.8 para fines de demostración. Si usa este método de filtrado, puede que necesite elegir un valor alfa diferente.

Los acelerómetros usan el sistema de coordenadas del sensor estándar. En la práctica, esto significa que las siguientes condiciones se aplican cuando un dispositivo está tumbado sobre una mesa en su orientación natural:

- Si empujamos el dispositivo por la izquierda (para que se mueva hacia la derecha), el valor de la aceleración "x" es positivo.
- Si empujamos el dispositivo por abajo (para que se aleje de nosotros), el valor de aceleración "y" es positivo.
- Si empujamos el dispositivo hacia el cielo con una aceleración de $(A \sim m/s^2)$, el valor de aceleración "z" es $(A + 9.81)$, que corresponde a la aceleración del dispositivo $(A \sim m/s^2)$ menos la fuerza de la gravedad $(-9.81 \sim m/s^2)$.
- El dispositivo estacionario o parado tendrá un valor de aceleración en "z" de $(+9.81)$, que corresponde a la aceleración del dispositivo $(0 \sim m/s^2)$ menos la fuerza de la gravedad, que es $(-9.81 \sim m/s^2)$.

En general, el acelerómetro es un buen sensor para usar si monitorizamos el

movimiento del dispositivo. Casi todos los teléfonos y tabletas con Android tienen un acelerómetro, y usan aproximadamente 10 veces menos energía que los otros sensores de movimiento. Una desventaja es que podríamos tener que implementar filtros de paso bajo y paso alto para eliminar las fuerzas gravitatorias y reducir el ruido.

9.- Giroscopio.

El giroscopio mide la velocidad de rotación en rad/s (radianes por segundo) alrededor del eje x, y y z de un dispositivo. El siguiente código muestra cómo obtener una instancia del giroscopio predeterminado:

Código

```
1 private SensorManager mSensorManager;  
2 private Sensor mSensor;  
3  
4 ...  
5 mSensorManager = (SensorManager) getSystemService(Context.SENSOR_S  
    mSensor = mSensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE);
```

El sistema de coordenadas del sensor es el mismo que el utilizado para el sensor de aceleración. La rotación es positiva en el sentido contrario a las agujas del reloj; es decir, un observador que mira desde una ubicación positiva en los ejes x, y o z en un dispositivo colocado en el origen informaría la rotación positiva si el dispositivo parecía estar girando en sentido contrario a las agujas del reloj. Esta es la definición matemática estándar de rotación positiva y no es lo mismo que la definición de balanceo que utiliza el sensor de orientación.

Normalmente, la salida del giroscopio se integra a lo largo del tiempo para calcular una rotación que describe el cambio de ángulos sobre el paso de tiempo.

Ejemplo

```
1 // Create a constant to convert nanoseconds to seconds.  
2 private static final float NS2S = 1.0f / 1000000000.0f;  
3  
4 private final float[] deltaRotationVector = new float[4]();  
5 private float timestamp;  
6  
7  
8 public void onSensorChanged(SensorEvent event) {  
9     // This timestep's delta rotation to be multiplied by the current  
10     // after computing it from the gyro sample data.  
11     if (timestamp != 0) {  
12         final float dT = (event.timestamp - timestamp) * NS2S;
```

```

// Axis of the rotation sample, not normalized yet.
float axisX = event.values[0];
float axisY = event.values[1];
float axisZ = event.values[2];

// Calculate the angular speed of the sample
float omegaMagnitude = sqrt(axisX*axisX + axisY*axisY + axisZ*axisZ);

// Normalize the rotation vector if it's big enough to get the angle
// (that is, EPSILON should represent your maximum allowable magnitude)
if (omegaMagnitude > EPSILON) {
    axisX /= omegaMagnitude;
    axisY /= omegaMagnitude;
    axisZ /= omegaMagnitude;
}

// Integrate around this axis with the angular speed by the time delta
// in order to get a delta rotation from this sample over the time delta
// We will convert this axis-angle representation of the delta rotation
// into a quaternion before turning it into the rotation matrix
float thetaOverTwo = omegaMagnitude * dT / 2.0f;
float sinThetaOverTwo = sin(thetaOverTwo);
float cosThetaOverTwo = cos(thetaOverTwo);
deltaRotationVector[0] = sinThetaOverTwo * axisX;
deltaRotationVector[1] = sinThetaOverTwo * axisY;
deltaRotationVector[2] = sinThetaOverTwo * axisZ;
deltaRotationVector[3] = cosThetaOverTwo;
}

timestamp = event.timestamp;
float[] deltaRotationMatrix = new float[9];
SensorManager.getRotationMatrixFromVector(deltaRotationMatrix, deltaRotationVector);
// User code should concatenate the delta rotation we computed with
// the current rotation in order to get the updated rotation.
// rotationCurrent = rotationCurrent * deltaRotationMatrix;
}

```

Los giroscopios estándar brindan datos rotativos brutos sin ningún filtrado o corrección por ruido y deriva (sesgo). En la práctica, el ruido y la deriva del giroscopio introducirán errores que deben ser compensados.



Autoevaluación

El sensor de aceleración...

- ☐ es un buen sensor para usar si monitorizamos el movimiento del dispositivo.
- ☐ se usa para determinar la orientación relativa del dispositivo en el espacio.
- ☐ mide la velocidad de rotación.

Opción correcta

Incorrecto

Incorrecto

Solución

1. Opción correcta (#answer-408_144)
2. Incorrecto (#answer-408_10456)
3. Incorrecto (#answer-408_10459)



Autoevaluación

El sensor de gravedad...

- ☐ es un buen sensor para usar si monitorizamos el movimiento del dispositivo.
- ☐ se usa para determinar la orientación relativa del dispositivo en el espacio.

☐ mide la velocidad de rotación.

Incorrecto

Opción correcta

Incorrecto

Solución

1. Incorrecto (#answer-409_144)
2. Opción correcta (#answer-409_10462)
3. Incorrecto (#answer-409_10465)

10.- Sensor de aceleración lineal.

El sensor de aceleración lineal le proporciona un vector tridimensional que representa la aceleración a lo largo de cada eje del dispositivo, excluyendo la gravedad. Puede usar este valor para realizar la detección de gestos. El valor también puede servir como entrada para un sistema de navegación inercial, que utiliza la navegación a estima. El siguiente código muestra cómo obtener una instancia del sensor de aceleración lineal predeterminado:

Código

```
1 | private SensorManager mSensorManager;  
2 | private Sensor mSensor;  
3 |  
4 | ...  
5 | mSensorManager = (SensorManager) getSystemService(Context.SENSOR_SE  
   mSensor = mSensorManager.getDefaultSensor(Sensor.TYPE_LINEAR_ACCELE
```

Conceptualmente, este sensor le proporciona datos de aceleración según la siguiente relación:

$$\text{aceleración lineal} = \text{aceleración} - \text{aceleración debido a la gravedad}$$

Normalmente utilizaremos este sensor cuando deseemos obtener datos de aceleración sin la influencia de la gravedad. Por ejemplo, podría usar este sensor para saber lo rápido que va mi automóvil. El sensor de aceleración lineal siempre tiene un desplazamiento, que debe eliminar. La forma más sencilla de hacerlo es crear un paso de calibración en nuestra aplicación. Durante la calibración, podemos pedirle al usuario que configure el dispositivo en una mesa, y luego lea los desplazamientos para los tres ejes. A continuación, deberíamos restar ese desplazamiento de las lecturas directas del sensor de aceleración para obtener la aceleración lineal real.

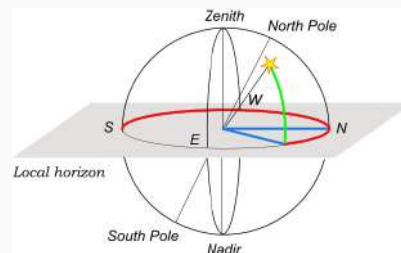
11.- Sensor de campo geomagnético.

El sensor de campo geomagnético le permite controlar los cambios en el campo magnético de la tierra. El siguiente código muestra cómo obtener una instancia del sensor de campo geomagnético predeterminado:

Código

```
1 private SensorManager mSensorManager;  
2 private Sensor mSensor;  
3  
4 ...  
5 mSensorManager = (SensorManager) getSystemService(Context.SENSOR_SE  
mSensor = mSensorManager.getDefaultSensor(Sensor.TYPE_MAGNETIC_FIEL
```

Este sensor proporciona datos de intensidad de campo sin procesar (en μT) para cada uno de los tres ejes de coordenadas. Por lo general, no es necesario que usemos este sensor directamente. En su lugar, podemos usar el sensor de vector de rotación para determinar el movimiento de rotación sin procesar o podemos usar el acelerómetro y el sensor de campo geomagnético junto con el método **getRotationMatrix()** para obtener la matriz de rotación y la matriz de inclinación. A continuación, podemos utilizar estas matrices con los **getOrientation()** y **getInclination()** para obtener datos de [inclinación azimutal](#)  y geomagnética.



Q [Francisco Javier Blanco González](#) (CC BY-SA)

Al probar nuestra aplicación, podemos mejorar la precisión del sensor agitando el dispositivo describiendo un patrón con forma de 8.

12.- Sensor de proximidad.

El sensor de proximidad nos permite determinar lo lejos que está un objeto del dispositivo. El siguiente código muestra cómo obtener una instancia del sensor de proximidad predeterminado:

Código

```
1 private SensorManager mSensorManager;  
2 private Sensor mSensor;  
3  
4 ...  
5 mSensorManager = (SensorManager) getSystemService(Context.SENSOR_SE  
    mSensor = mSensorManager.getDefaultSensor(Sensor.TYPE_PROXIMITY);
```

El sensor de proximidad se usa generalmente para determinar lo lejos que está la cabeza de una persona del sensor del de proximidad, el cual suele estar junto al auricular del teléfono (por ejemplo, cuando un usuario realiza o recibe una llamada telefónica). La mayoría de los sensores de proximidad devuelven la distancia absoluta, en cm, pero algunos devuelven solo valores cercanos y lejanos.

Ejemplo

El siguiente código muestra cómo usar el sensor de proximidad:

```
1 public class SensorActivity extends Activity implements SensorEvent  
2  
3  
4 private SensorManager mSensorManager;  
5 private Sensor mProximity;  
6  
7  
8 @Override  
9  
10 public final void onCreate(Bundle savedInstanceState) {  
11     super.onCreate(savedInstanceState);  
12     setContentView(R.layout.main);  
13  
14  
15     // Get an instance of the sensor service, and use that to get an  
16     // a particular sensor.  
17  
18     mSensorManager = (SensorManager) getSystemService(Context.SENSOR_  
19  
20
```

```

        mProximity = mSensorManager.getDefaultSensor(Sensor.TYPE_PROXIMITY);
    }

    @Override
    public final void onAccuracyChanged(Sensor sensor, int accuracy) {
        // Do something here if sensor accuracy changes.
    }

    @Override
    public final void onSensorChanged(SensorEvent event) {
        float distance = event.values[0];
        // Do something with this sensor data.
    }

    @Override
    protected void onResume() {
        // Register a listener for the sensor.
        super.onResume();
        mSensorManager.registerListener(this, mProximity, SensorManager.S
    }

    @Override
    protected void onPause() {
        // Be sure to unregister the sensor when the activity pauses.
        super.onPause();
        mSensorManager.unregisterListener(this);
    }
}

```

Algunos sensores de proximidad devuelven valores binarios que representan "cerca" o "lejos". En este caso, el sensor generalmente informa su valor máximo de rango en el estado lejano y un valor menor en el estado cercano. Normalmente, el valor lejano tiene un valor > 5 cm, pero esto puede variar de un sensor a otro. Podemos determinar el rango máximo de un sensor utilizando el método **getMaximumRange()**.



Autoevaluación

El sensor de aceleración lineal se usa para la detección de gestos o para calcular por ejemplo la velocidad a la que viaja un vehículo.

☐ Verdadero ☐ Falso

Verdadero

13.- Ejercicio resuelto 2.



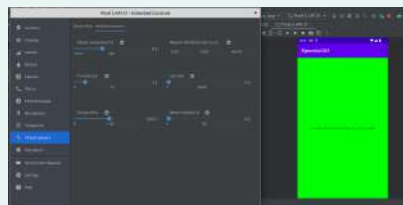
Ejercicio Resuelto

Crear una aplicación que utilice el sistema sensor de proximidad del dispositivo, de forma que teniendo un color de fondo, éste cambia dependiendo de si se sobrepasa una distancia concreta a dicho sensor.

Si entramos en la funcionalidad de sensores que tiene la emulación, y vamos dentro de los sensores adicionales (Additional sensors) al sensor de proximidad (Proximity) podemos marcar la distancia a la que virtualmente nos acercamos o alejamos, desplazando el scroll a valores menores o mayores de 1 cm.

En el caso de estar por encima de este umbral se mostrará la pantalla de color verde (figura 1) y en caso contrario (distancia menor de 1 cm) se activará el color rojo en la pantalla (figura 2).

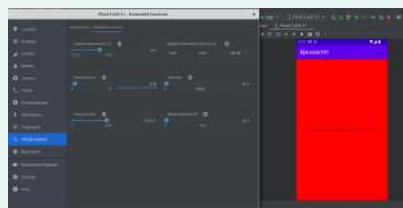
Figura 1. Distancia > 1 cm.



[\(p1001-img001.png\)](#)

[Google Developers \(https://
developer.android.com/studio\)](https://developer.android.com/studio) (Uso educativo
nc)

Figura 2. Distancia < 1 cm.



[\(p1001-img002.png\)](#)

[Google Developers \(https://
developer.android.com/studio\)](https://developer.android.com/studio) (Uso educativo
nc)

Mostrar retroalimentación

AndroidManifest.xml (#exe-accordion-20-0)

Añadimos la siguiente línea en el archivo de manifiesto de nuestra aplicación:

```
1 | <uses-feature android:name="android.hardware.sensor
```

activity_main.xml (#exe-accordion-21-0)


```

1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout
3      xmlns:app="http://schemas.android.com/apk/res-aut
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      tools:context=".MainActivity">
8
9      <TextView
10         android:layout_width="wrap_content"
11         android:layout_height="wrap_content"
12         android:text="Cambio de color de fondo por proxim
13         app:layout_constraintBottom_toBottomOf="parent"
14         app:layout_constraintLeft_toLeftOf="parent"
15         app:layout_constraintRight_toRightOf="parent"
16         app:layout_constraintTop_toTopOf="parent" />
17
18  </androidx.constraintlayout.widget.ConstraintLayout

```

MainActivity.java (#exe-accordion-22-0)

```

1  package com.cidead.pmdm.ejercicio101;
2
3
4  import androidx.appcompat.app.AppCompatActivity;
5
6  import android.graphics.Color;
7  import android.hardware.Sensor;
8  import android.hardware.SensorEvent;
9  import android.hardware.SensorEventListener;
10
11
12
13

```

```

import android.hardware.SensorManager;

import android.os.Bundle;

import android.widget.Toast;


public class MainActivity extends AppCompatActivity {

    private SensorManager sensorManager;

    private Sensor sensorProximidad;

    private SensorEventListener escuchadorSensorProximidad;


    @Override

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        sensorManager = (SensorManager) getSystemService(SENSOR_SERVICE);

        sensorProximidad = sensorManager.getDefaultSensor(Sensor.TYPE_PROXIMITY);

        if(sensorProximidad==null){
            Toast.makeText(this, "Sensor de proximidad no disponible", Toast.LENGTH_SHORT).show();
            finish();
        }


        escuchadorSensorProximidad=new SensorEventListener() {

            @Override

            public void onSensorChanged(SensorEvent sensorEvent) {
                if(sensorEvent.values[0] < sensorProximidad.getMaximumRange()){
                    getWindow().getDecorView().setBackgroundColor(Color.parseColor("#FF0000"));
                }
                else {
                    getWindow().getDecorView().setBackgroundColor(Color.parseColor("#00FF00"));
                }
            }

            @Override

            public void onAccuracyChanged(Sensor sensor, int type) {}

        };

        sensorManager.registerListener(escuchadorSensorProximidad, sensorProximidad, SensorManager.SENSOR_DELAY_NORMAL);
    }
}

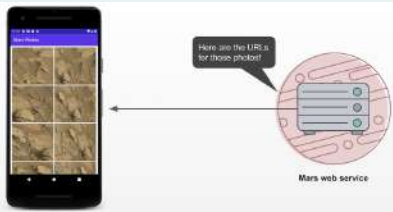
```

```
@Override
protected void onPause(){
super.onPause();
sensorManager.unregisterListener(escuchadorSensorP
}
}
```

III - Web Services.



Caso práctico



(PMDM11_CONT_R1_WebServices.png)

Q [Google Developers \(https://
developer.android.com/guide/topics/sensors/
gnss?hl=es-419\)](https://developer.android.com/guide/topics/sensors/gnss?hl=es-419) (Uso educativo nc)

Una de las aplicaciones que BK Programación ha desarrollado es para un cliente con presencia en distintos países del mundo. Esto implica que además del factor multidioma hay que tener en cuenta otra cuestión importante como la monedas -divisas- utilizadas en su tienda.

- Tenemos un cliente con tienda en distintos países del mundo -explica Ada a su equipo-, y necesita que se pueda comprar en su tienda en la divisa que corresponda a cada país.

- Eso es un problema -razona Juan-, puesto que el valor de la divisa fluctúa cada día. Para saber cuánto cuesta un producto de nuestro cliente en distintas monedas, necesitaríamos dedicar varios minutos cada día (sin excepción) para calcular, por ejemplo, cuántos dólares (USD) cuestan los productos fabricados en euros (EUR). Pero seguro que ya tienes una solución al problema y nos la vas a mostrar.

- ¡Por supuesto! -exclama Ada-. No se puede dedicar tiempo cada día a realizar la conversión, debe ser algo automático. Por ello es necesario que nuestra aplicación se conecte con algún servicio que ofrezca esta información publicada en Internet. El mecanismo normal más extendido actualmente es publicar esta información siguiendo unos estándares de llamadas que usan el protocolo HTTP y responden la información usando formatos como el XML o el JSON: estamos hablando de REST.

- ¿Y de este modo se puede acceder sólo a texto publicado? -pregunta María.

- No -responde Ada-. Además de formato texto, es posible también publicar la URL de distintas imágenes.

En la figura se ve cómo la NASA, por ejemplo, publica imágenes capturadas por sus robots en Marte.

El equipo se dispone a trabajar con distintas librerías como Retrofit que, con conocimientos de programación orientada a objetos (Clases, POJO, etc.), facilitarán la labor de hacer aplicaciones Android más y mejor conectadas.

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

1.- HttpURLConnection.

HttpURLConnection es una clase específica que a través del protocolo HTTP permite a los sistemas clientes (habitualmente navegadores, aunque en nuestro caso, el propio dispositivo Android será el cliente sin tener que mediar un navegador para acceder al contenido) acceder al contenido de páginas web.

Los navegadores hacen uso de la primitiva o método **GET** de HTTP para descargarse una página web del servidor. Es decir, cuando en un navegador web cargamos la URL: `https://dominio.com/dir/pagina.html`, internamente se está haciendo un `GET dominio.com/dir/pagina.html`. El servidor recibe esta petición y envía al cliente la página en formato HTML.

De la misma forma, nuestro dispositivo Android, con las librerías adecuadas, será capaz de reproducir directamente el proceso anterior. Dichas librerías principalmente son de dos grupos, las librerías **java.net.*** y las librerías **org.apache.commons.httpclient.***.

Veremos un ejemplo de uso de HttpURLConnection, que pertenece al primer grupo de las librerías anteriores. Concretamente, usaremos las clases:

- `java.net.HttpURLConnection`
- `java.net.URL`
- `java.net.URLEncoder`

1.1.- Ejercicio resuelto 1 (HttpURLConnection).



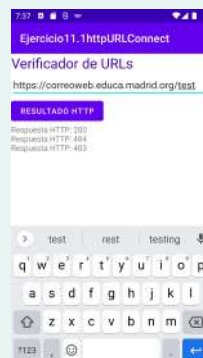
Ejercicio Resuelto

Realizaremos una aplicación que solicitará al usuario una URL (incluyendo el protocolo) y se conectará a ella para devolver el resultado HTTP de la misma. Te mostramos en la imagen cómo quedaría la aplicación con estos ejemplos:

- `https://www.educa2.madrid.org/educamadrid/"` OK 200
- `https://www.educa2.madrid.org/educamadridNO/"` No encontrado 404
- `https://correoweb.educa.madrid.org/test` Forbidden 403

Es importante que conozcas los distintos [códigos de respuesta del protocolo HTTP](#).

NOTA: Debes tener en cuenta, que en ocasiones, dependiendo de la calidad del certificado del servidor al que te conectes puedes recibir códigos de estado no esperados.



(p1101-img001.png)

[Google Developers](#)

([https://](#)

[developer.android.com/](#)

[studio](#)).(Uso educativo nc)

Mostrar retroalimentación

AndroidManifest.xml (#exe-accordion-23-0)

Lo primero será dar permiso de uso de Internet a través del **AndroidManifest.xml** mediante la directiva:

```
1 | <uses-permission android:name="android.permission.I
```

activity_main.xml (#exe-accordion-24-0)

Además crearemos una interfaz básica a través de **activity_main.xml**:


```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android
3      xmlns:app="http://schemas.android.com/apk/res-a
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:orientation="vertical"
8      tools:context="com.cidead.pmdm.ejercicio111http
9
10
11
12
13
14  <TextView
15      android:layout_width="wrap_content"
16      android:layout_height="wrap_content"
17      android:text="Verificador de URLs"
18      android:textSize="25dp"
19      android:textColor="@color/purple_700"
20      android:textAlignment="center"/>
21
22
23
24
25
26  <EditText android:id="@+id/et1"
27      android:layout_height="wrap_content"
28      android:layout_width="match_parent"
29      android:hint="URL a verificar"
30      android:text="https://www.educa2.madrid.org
31
32
33
34  <Button android:id="@+id/btn1"
35      android:layout_height="wrap_content"
36      android:layout_width="wrap_content"
37      android:onClick="buscar"
38      android:text="Resultado HTTP"/>
39
40
41  <TextView android:id="@+id/tv1"
42      android:layout_height="match_parent"
43      android:layout_width="match_parent"/>
44
45  </LinearLayout>
```

MainActivity.java (#exe-accordion-25-0)

Por último, crearemos el **MainActivity.java**:

```
1 package com.cidead.pmdm.ejercicio11httpurlconnect;
2
3
4 import androidx.appcompat.app.AppCompatActivity;
5
6
7 import android.os.Bundle;
8 import android.os.StrictMode;
9 import android.util.Log;
10 import android.view.View;
11 import android.widget.EditText;
12 import android.widget.TextView;
13 import java.net.HttpURLConnection;
14 import java.net.URL;
15
16
17 public class MainActivity extends AppCompatActivity {
18
19     private EditText et1;
20     private TextView tv1;
21
22     @Override
23
24     protected void onCreate(Bundle savedInstanceState) {
25         super.onCreate(savedInstanceState);
26         setContentView(R.layout.activity_main);
27         et1 = (EditText) findViewById(R.id.et1);
28         tv1 = (TextView) findViewById(R.id.tv1);
29         //Con la siguiente línea permitiremos que s
30         StrictMode.setThreadPolicy(new StrictMode.T
31     }
32
33     public void buscar(View view) {
34         try {
35             String urlBusqueda = et1.getText().toSt
36             String resultado = resultadosHTTP(urlBu
```

```

        tv1.append("Respuesta HTTP: " + resultado);
    } catch (Exception e) {
        tv1.append("Error al conectar\n");
        Log.e("HTTP", e.getMessage(), e);
    }
}

```

```

String resultadosHTTP(String urlABuscar) throws Exception {
    String resultado = "No conectado";
    try {
        //URL url = new URL("https://google.es/");
        URL url = new URL(urlABuscar);
        //Preparamos la conexión con la clase HttpURLConnection
        HttpURLConnection conexion = (HttpURLConnection) url.openConnection();
        //Establecemos cabeceras HTTP. Podemos usar la cabecera "User-Agent" para
        //simular un navegador. Por ejemplo: "Mozilla/5.0 (Windows NT 6.1)", pero
        //podemos usar también "Chrome/30.0.1599.101".
        conexion.setRequestProperty("User-Agent", "Mozilla/5.0 (Windows NT 6.1)");
        //Con el método getResponseCode() establecemos el código de respuesta.
        resultado = String.valueOf(conexion.getResponseCode());
    } catch (Exception e) {
        tv1.append("Error en la URL\n");
        Log.e("HTTP", e.getMessage(), e);
    }
    return resultado;
}
}

```

2.- REST.

De la Wikipedia:

La **transferencia de estado representacional** (en inglés representational state transfer) o **ReST** es un estilo de arquitectura software para sistemas hipermedia distribuidos como la World Wide Web. El término se originó en el año 2000, en una tesis doctoral sobre la web escrita por Roy Fielding, uno de los principales autores de la especificación del protocolo HTTP, y ha pasado a ser ampliamente utilizado por la comunidad de desarrollo.

REST describe cualquier interfaz entre sistemas que utilice directamente HTTP para obtener datos o indicar la ejecución de operaciones sobre los datos en cualquier formato (XML, JSON, etc.), sin las abstracciones adicionales de los protocolos basados en patrones de intercambio de mensajes, como por ejemplo [SOAP](#) .

REST afirma que la web ha disfrutado de escalabilidad como resultado de una serie de diseños fundamentales clave:

- Un protocolo **cliente/servidor sin estado**: cada mensaje HTTP contiene toda la información necesaria para comprender la petición. Como resultado, ni el cliente ni el servidor necesitan recordar ningún estado de las comunicaciones entre mensajes. Sin embargo, en la práctica, muchas aplicaciones basadas en HTTP utilizan cookies y otros mecanismos para mantener el estado de la sesión (algunas de estas prácticas, como la reescritura de URLs, no son permitidas por REST)
- Un conjunto de **operaciones bien definidas** que se aplican a todos los recursos de información: HTTP en sí define un conjunto pequeño de operaciones, las más importantes son **POST**, **GET**, **PUT** y **DELETE**. Con frecuencia estas operaciones se equiparan a las operaciones **CRUD** en bases de datos (CLAB en castellano: crear, leer, actualizar, borrar) que se requieren para la persistencia de datos, aunque POST no encaja exactamente en este esquema.
- Una **sintaxis universal** para identificar los recursos. En un sistema REST, cada recurso es direccionable únicamente a través de su URI.
- El uso de **hipermedios**, tanto para la información de la aplicación como para las transiciones de estado de la aplicación: la representación de este estado en un sistema REST son típicamente **HTML** o **XML**. Como resultado de esto, es posible navegar de un recurso REST a muchos otros simplemente siguiendo enlaces sin requerir el uso de registros u otra infraestructura adicional



Debes conocer

Existen diversos mecanismos para consumir un servicio web. Los más popularizados son SOAP y REST. Por simplificación, en esta unidad veremos solamente REST, aunque es importante que conozcas las diferencias entre ambos mecanismos, puede que en algún momento tengas que utilizarlas.

Las diferencias principales entre SOAP y REST son:

- SOAP significa Protocolo simple de acceso a objetos, mientras que REST significa Transferencia de estado representacional.
- SOAP es un protocolo mientras que REST es un patrón arquitectónico.
- SOAP usa interfaces de servicio para exponer su funcionalidad a las aplicaciones cliente, mientras que REST usa localizadores de servicios uniformes para acceder a los componentes en el dispositivo de hardware.
- SOAP necesita más ancho de banda para su uso, mientras que REST no necesita mucho ancho de banda.
- Comparando SOAP vs REST API, SOAP solo funciona con formatos XML, mientras que REST funciona con texto sin formato, XML, HTML y JSON.

SOAP no puede hacer uso de REST mientras que REST puede hacer uso de SOAP.

2.1.- Ejercicio resuelto 2 (Fuentes de datos REST).

Existen múltiples fuentes de datos que proporcionan servicio web mediante REST.

Una de ellas es: <https://openexchangerates.org> 

Si nos conectamos a dicha URL (figura 1), bajando en esa ventana vemos que hay un botón para obtener un ID de uso, ya que es habitual que para evitar abusos de utilización haya que registrarse (algunos otros servicios no requieren identificación ni API key) (figura 2). Utilizaremos el plan gratuito (figura 3).

Figura 1. Acceso Open Exchange Rates



Q [Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 2. Obtención de ID.



Q [Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 3. Plan gratuito.



Q [Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

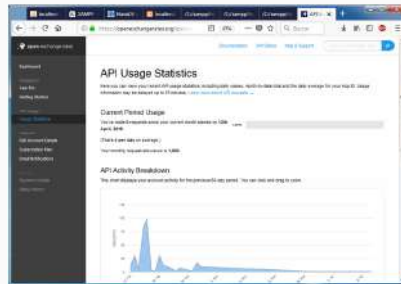
Nos registramos (figura 4). Una vez validados, podemos entrar en nuestro escritorio (figura 5) y buscar la App ID que tenemos (figura 6).

Figura 4. Registro.



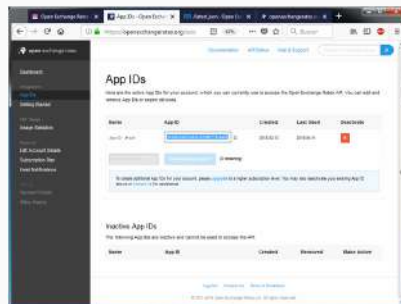
Q [Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 5. Acceso a escritorio.



Q [Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 6. Búsqueda de la App ID



Q [Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Ahora tendremos que buscar *API Endpoints*. Para ello, vamos al enlace "*Getting Started*" de la ventana anterior, abriéndose la ventana mostrada en la figura 7. De entre las *API EndPoints* cogemos, por ejemplo, la primera (*/latest.json*) (figura 8). Dicha página describe como se utiliza (definición y parámetros que admite) dicha *API EndPoint*.

Figura 7. Enlace "Getting Started"



Q [Openexchangerates.org](https://openexchangerates.org) (Usa educativo nc)

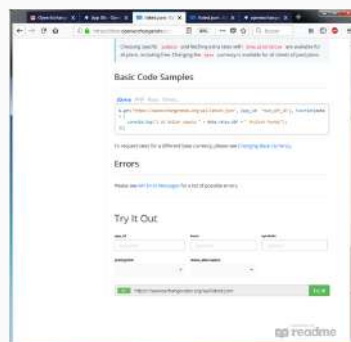
Figura 8. Seleccionamos /latest.json



Q [Openexchangerates.org](https://openexchangerates.org) (Usa educativo nc)

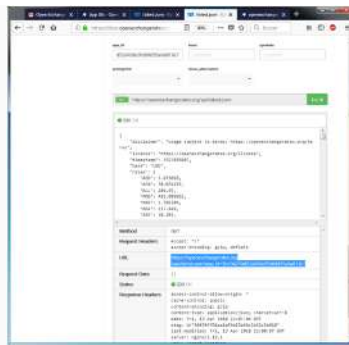
En la parte de abajo podemos probarla, para lo cual debemos suministrar obligatoriamente la App ID anteriormente mencionada (figura 9) y darle a "Try It!" (figura 10):

Figura 9. Suministrar la App ID



Q [Openexchangerates.org](https://openexchangerates.org) (Usa educativo nc)

Figura 10. Try it!



Q [Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Podemos ver que se genera la última valoración (ver que la fecha coincide con la del día actual en el que hacemos la petición) de cambio del dólar (base) con respecto a las demás monedas, usando formato **JSON**.

Además podemos poner la URL en un navegador e igualmente aparecerán los resultados en formato **JSON** (figura 11). Dicho fichero será descargable, pero en todo caso será preferible mantener como fuente de datos al servidor web de openexchangerates.org para asegurarnos la puntual actualización de los datos.

Si ahora intentamos cambiar la base a euros (EUR), vemos que desafortunadamente nos da un mensaje de error diciendo que el cambio de base está reservado a planes de pago, por lo cual el único cambio de base que podemos hacer es el de dólar americano (USD) a todo lo demás (figura 12).

En todo caso, este problema no es insalvable ya que será posible hacer el cambio a través de un simple algoritmo matemático, partiendo del cambio de dólar americano a todo lo demás, para poder hacer cualquier conversión posible.

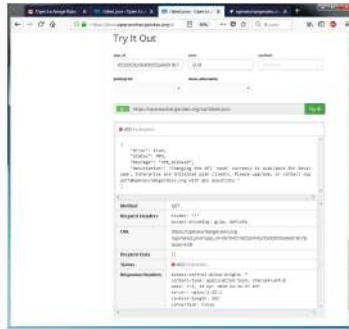
Podría igualmente llevar la URL anterior a aplicaciones como SoapUI para probar el acceso al Webservice anterior (figura 13).

Figura 11. Resultados en formato JSON.



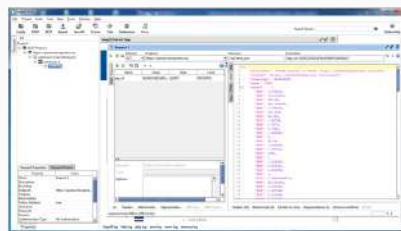
Q [Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 12. Cambio de la base a euros.



Q [Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Figura 13. Uso de SoapUI.



Q [Openexchangerates.org](https://openexchangerates.org) (Uso educativo nc)

Como vemos, este es un ejemplo de como somos capaces de acceder a servicios REST provistos por web services externos. Hay muchos más casos, solo es cuestión de investigar en Internet. En todo caso, el consumo de estos web services es algo complicado, pero gracias a ciertas librerías que veremos más adelante, el acceso y manipulación de datos será más sencillo. Concretamente veremos Retrofit.

3.- Consumo de un Web Service.

Un web service por tanto no es más que una forma de estructurar un conjunto de acciones que permiten a las aplicaciones interactuar con un servidor para consultar datos o modificarlos mediante los métodos del protocolo HTTP.

Para desarrollar la parte servidor (el web service en sí) puede usarse cualquier tecnología capaz de publicar información mediante HTTP (PHP, ASP.NET, Java + TOMAT, etc.). Para "usar" o "consumir" este web service desarrollado con REST es necesario establecer conexiones HTTP mediante las aplicaciones. Es decir, las aplicaciones se conectan a Internet (sin usar un navegador) y son capaces de descargar e incluso subir datos al servidor.

Web service en el servidor

Para desarrollar la parte de servidor existen herramientas web sobradamente preparadas que proporcionan un servicio web completamente fiable. Estará especialmente indicado la instalación del paquete **XAMPP**, **WAMP**, o **LAMP**, dependiendo de la distribución y del sistema operativo. **XAMPP**, por ejemplo, es un paquete de software libre que proporciona una serie de aplicaciones de alto nivel, entre las cuales destaca el servidor web Apache, el sistema de gestión de bases de datos MySQL y los intérpretes para lenguajes de script PHP y Perl. El nombre proviene del acrónimo de las aplicaciones anteriormente indicadas para cualquier sistema operativo: X (se refiere a cualquiera de los diferentes sistemas operativos más utilizados), Apache, MariaDB/MySQL, PHP, Perl.

A partir de la versión 5.6.15, **XAMPP** cambió la base de datos **MySQL** por **MariaDB**, la cual es una versión actualizada de MySQL con licencia [GPL](#), y que surgió con el propósito de garantizar que los usuarios finales pudieran seguir haciendo uso de un sistema gestor de bases de datos avanzado, cosa que quedaba en entredicho por la amenaza de que **Oracle**, propietaria de MySQL pudiera comenzar a distribuir este SGBD bajo licencias no libres.

El programa proporciona un servidor web libre de altas prestaciones y muy fiable, fácil de usar y capaz de interpretar páginas dinámicas. Actualmente, XAMPP está disponible para Windows, Linux, y OS X. El XAMPP podemos instalarlo tanto en un equipo local, como en un sistema remoto.

Los certificados de servidor

Uno de los aspectos en los que más está evolucionando Android, según avanzan las APIs, es la seguridad. Esto ha hecho que desde hace tiempo las aplicaciones sólo tienen disponible conexiones a servidores HTTPS, es decir, a servidores que cuentan con un certificado que garantiza la encriptación de la comunicación. Conseguir esto en un servidor Apache es algo sencillo porque existen algunas formas de instalar lo que se denominan certificados autofirmados que permiten este cifrado en las comunicaciones. Sin embargo, las restricciones de Android desde la API 28 han aumentado, y ahora las aplicaciones no tienen habilitado el acceso a ningún servicio web que no ofrezca HTTPS con un certificado avalado por una [CA](#)  (autoridad certificadora) que asegure que el certificado pertenece a la empresa o persona que dice ser. Esto es necesario incluso cuando accedemos a direcciones IP de nuestra propia red o incluso a nuestra máquina anfitriona desde nuestro Android Studio.

Estas nuevas medidas complican la posibilidades de realizar aplicaciones que consuman servicios web desarrollados por nosotros mismos ya que es necesario instalar certificados **reales** en el servidor o crear entidades certificadoras, lo cual excede de los objetivos del módulo.



Autoevaluación

Sobre REST, indica las afirmaciones correctas...

- ☐ Puede usarse para conseguir datos de un servicio web e incluso subir datos al servidor con los métodos HTTP.
- ☐ No guarda estado.
- ☐ Devuelve siempre la información en formato JSON.
- ☐ En un sistema REST, cada recurso es direccionable mediante su URI.

Mostrar retroalimentación

Solución

1. Correcto (#answer-389_139)
2. Correcto (#answer-389_10805)
3. Incorrecto (#answer-389_10807)
4. Correcto (#answer-389_10809)

3.1.- Ejercicio resuelto 3 (Consumo de un Web Service).



Ejercicio Resuelto

A partir del web service público con el que trabajamos en la unidad anterior (<https://openexchangerates.org/api/latest.json>) que devuelve un conjunto de divisas y los valores de cambio de las mismas con respecto al dólar (USD) vamos a realizar una aplicación capaz de hablar con este servicio para que calcule el número de divisas disponibles.

Para realizar el ejercicio, la aplicación solicitará al usuario el ID de API que debes conseguir tal como se explicó en el ejercicio anterior. En caso de que este no sea correcto, la aplicación devolverá un mensaje de error a través de TOAST. En caso de que pueda conectar, la aplicación simplemente contará las divisas que tiene disponibles y mostrará el JSON devuelto por el servicio web.

Este es el aspecto que tendrá la aplicación antes y después de establecer la conexión con el servicio.

Antes de establecer la conexión con el servicio.



(p1103-img001.png)

Q [Google Developers](https://developer.android.com/studio)
([https://](https://developer.android.com/studio)
[developer.android.com/](https://developer.android.com/studio)
[studio](https://developer.android.com/studio)) (Uso educativo nc)

Después de establecer la conexión con el servicio.



(p1103-img002.png)

[Google Developers](https://developer.android.com/studio)

([https://
developer.android.com/
studio](https://developer.android.com/studio)) (Uso educativo nc)

Nota: Puedes encontrar cualquier imagen relacionada con las divisas en Internet para usarla como recurso "drawable".

Mostrar retroalimentación

AndroidManifest.xml (#exe-accordion-26-0)

```
1 | <uses-permission android:name="android.permission.I
```

activity_main.xml (#exe-accordion-27-0)

```
1 | <?xml version="1.0" encoding="utf-8"?>
2 | <LinearLayout xmlns:android="http://schemas.android
3 |
4 |     xmlns:app="http://schemas.android.com/apk/res-a
```

```
xmlns:tools="http://schemas.android.com/tools"  
android:layout_width="match_parent"  
android:layout_height="match_parent"  
android:orientation="vertical"  
tools:context="com.cidead.pmdm.ejercicio113.Ma
```

```
<TextView  
    android:id="@+id/tvTitulo"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:gravity="center"  
    android:text="Acceso a openexchangerates.or  
    android:textSize="20dp"  
    tools:text="API ID"  
    android:layout_margin="15dp"/>
```

```
<ImageView  
    android:id="@+id/imgvLogin"  
    android:layout_width="match_parent"  
    android:layout_height="80dp"  
    app:srcCompat="@drawable/divisas"  
    android:layout_margin="20dp"/>
```

```
<EditText  
    android:id="@+id/etAPIID"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:hint="Introduce tu API ID"/>
```

```
<Button  
    android:id="@+id/btnLogin"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Calcular divisas disponibles"  
    android:layout_margin="15dp"/>
```

```
<TextView  
    android:id="@+id/tvDivisasEncontradas"
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:textSize="20dp"
        android:layout_margin="15dp"/>

        <TextView
            android:id="@+id/tvJsonSent"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center"
            android:textSize="12dp"
            android:layout_margin="5dp"/>

    </LinearLayout>

```

MainActivity.java (#exe-accordion-28-0)

```

1  package com.cidead.pmdm.ejercicio113;
2
3
4  import androidx.appcompat.app.AppCompatActivity;
5
6
7  import android.content.Intent;
8  import android.os.Bundle;
9  import android.util.Log;
10 import android.view.View;
11 import android.widget.Button;
12 import android.widget.EditText;
13 import android.widget.TextView;
14 import android.widget.Toast;
15
16 import org.json.JSONArray;
17
18
19
20
21

```



```

import org.json.JSONObject;

import java.io.DataOutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.Scanner;

public class MainActivity extends AppCompatActivity {
    EditText etAPIID;
    TextView tvDivisasEncontradas, tvJsonSent;
    Button btnLogin;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        etAPIID=(EditText)findViewById(R.id.etAPIID);
        tvDivisasEncontradas =(TextView)findViewById(R.id.tvDivisasEncontradas);
        tvJsonSent = (TextView)findViewById(R.id.tvJsonSent);
        btnLogin=(Button)findViewById(R.id.btnLogin);
        btnLogin.setOnClickListener(new View.OnClickListener() {

            @Override
            public void onClick(View view) {
                //Creamos un hilo para conectar por internet
                Thread hilo=new Thread(){
                    @Override
                    public void run() {
                        super.run();
                        final String respuesta=enviarDatos(etAPIID.getText().toString());
                        runOnUiThread(new Runnable() {
                            @Override
                            public void run() {
                                //Limpiamos por si ya habia datos
                                tvDivisasEncontradas.setText("");
                                tvJsonSent.setText(respuesta);
                                //Toast.makeText(getApplicationContext(), "Se encontraron "+
                                int numElementosEnc

```

```

        if (numElementosEnc
            tvDivisasEncont
                "JSon d
            tvJsonSent.setT
        }
        else{
            Toast.makeText(
        }
    }
});
}
};
    hilo.start(); //Con esto arrancamos
}
});
}

```

```

public String enviarPost(String app_id){
    String parametros="app_id="+app_id; //Los
    HttpURLConnection conexion=null;
    String respuesta="";
    try{
        //Tenemos que consultar la página enviá
        URL url=new URL("https://openexchangerat
        conexion=(HttpURLConnection)url.openCon
        conexion.setRequestMethod("POST");
        conexion.setRequestProperty("Content-Le
        conexion.setDoOutput(true);
        DataOutputStream dos=new DataOutputStre
        dos.close();
        //Recogeremos la respuesta que nos devu
        Scanner lectura=new Scanner(conexion.ge
        while (lectura.hasNextLine())
            respuesta += lectura.nextLine();
    }catch (Exception e){
        Log.e("Error",e.toString());
    }
    return respuesta.toString();
}

```

```

    }

    //Creamos un método para contar cuantas divisas
    //Pasaremos como parámetro la respuesta que da
    public int objetoJSON(String respuesta){
        int numDivisas = 0;
        try{
            JSONObject jsonObject = new JSONObject(res
            JSONObject divisas = jsonObject.getJSONObject("divisas");
            numDivisas = divisas.length();
            //Log.d("Divisas", Integer.toString(numDivisas));
        }catch (Exception e){
            Log.e("Error", e.getMessage());
        }
        return numDivisas;
    }
}

```

Este es el texto JSON devuelto por el servicio
 (#exe-accordion-29-0)

```

1  {
2      "disclaimer": "Usage subject to terms: https://openexchangerates.org/terms",
3      "license": "https://openexchangerates.org/license",
4      "timestamp": 1654365613,
5      "base": "USD",
6      "rates": {
7          "AED": 3.6731,
8          "AFN": 89.162943,
9          "ALL": 112.223267,
10         "AMD": 447.269866,
11         "ANG": 1.805741,
12
13
14
15
16
17

```

"AOA": 426.1758,
"ARS": 120.67334,
"AUD": 1.387465,
"AWG": 1.8005,
"AZN": 1.7,
"BAM": 1.824519,
"BBD": 2,
"BDT": 89.20903,
"BGN": 1.821413,
"BHD": 0.37697,
"BIF": 2060.263278,
"BMD": 1,
"BND": 1.376531,
"BOB": 6.898421,
"BRL": 4.775,
"BSD": 1,
"BTC": 0.000033644322,
"BTN": 77.768691,
"BWP": 11.999496,
"BYN": 3.38321,
"BZD": 2.019646,
"CAD": 1.25957,
"CDF": 2004.038697,
"CHF": 0.961757,
"CLF": 0.029469,
"CLP": 811.81,
"CNH": 6.657479,
"CNY": 6.6603,
"COP": 3781.3558,
"CRC": 684.124098,
"CUC": 1,
"CUP": 25.75,
"CVE": 103.3,
"CZK": 23.0091,
"DJF": 178.372548,
"DKK": 6.9387,
"DOP": 55.207702,
"DZD": 145.41,

"EGP": 18.630186,
"ERN": 15.000001,
"ETB": 52.074408,
"EUR": 0.932923,
"FJD": 2.1368,
"FKP": 0.800737,
"GBP": 0.800737,
"GEL": 2.965,
"GGP": 0.800737,
"GHS": 7.840332,
"GIP": 0.800737,
"GMD": 54.15,
"GNF": 8870.767996,
"GTQ": 7.710011,
"GYD": 209.623421,
"HKD": 7.8448,
"HNL": 24.622892,
"HRK": 7.0142,
"HTG": 114.038788,
"HUF": 365.63,
"IDR": 14433.5,
"ILS": 3.335,
"IMP": 0.800737,
"INR": 77.6934,
"IQD": 1462.353565,
"IRR": 42350,
"ISK": 129.03,
"JEP": 0.800737,
"JMD": 154.180666,
"JOD": 0.7102,
"JPY": 130.845,
"KES": 116.978189,
"KGS": 79.525751,
"KHR": 4068.935217,
"KMF": 458.749854,
"KPW": 900,
"KRW": 1251.53,
"KWD": 0.306277,

"KYD": 0.834911,
"KZT": 435.599831,
"LAK": 13390.901035,
"LBP": 1514.968721,
"LKR": 361.204622,
"LRD": 151.999978,
"LSL": 15.510228,
"LYD": 4.786048,
"MAD": 9.878728,
"MDL": 19.067231,
"MGA": 4055.425659,
"MKD": 57.492275,
"MMK": 1855.118735,
"MNT": 3105.3554,
"MOP": 8.095189,
"MRU": 36.541135,
"MUR": 43.150001,
"MVR": 15.435,
"MWK": 816.25,
"MXN": 19.54515,
"MYR": 4.391,
"MZN": 63.850001,
"NAD": 15.49,
"NGN": 415.814006,
"NIO": 35.919846,
"NOK": 9.43645,
"NPR": 124.429799,
"NZD": 1.535745,
"OMR": 0.385029,
"PAB": 1,
"PEN": 3.713443,
"PGK": 3.530509,
"PHP": 52.909995,
"PKR": 198.136548,
"PLN": 4.28225,
"PYG": 6861.311273,
"QAR": 3.641,
"RON": 4.611,

"RSD": 109.539103,
"RUB": 63.360006,
"RWF": 1022.445353,
"SAR": 3.750665,
"SBD": 8.116969,
"SCR": 14.330625,
"SDG": 455.5,
"SEK": 9.7671,
"SGD": 1.3761,
"SHP": 0.800737,
"SLL": 12887.2,
"SOS": 579.587715,
"SRD": 21.22,
"SSP": 130.26,
"STD": 22994,
"STN": 23.15,
"SVC": 8.767037,
"SYP": 2512.53,
"SZL": 15.512281,
"THB": 34.29387,
"TJS": 11.306259,
"TMT": 3.51,
"TND": 3.02,
"TOP": 2.299263,
"TRY": 16.4176,
"TTD": 6.791608,
"TWD": 29.375,
"TZS": 2328,
"UAH": 29.457448,
"UGX": 3745.800949,
"USD": 1,
"UYU": 40.022165,
"UZS": 11015.948404,
"VES": 5.11435,
"VND": 23192,
"VUV": 114.629832,
"WST": 2.612952,
"XAF": 611.957277,

"XAG": 0.04559972,

"XAU": 0.00054022,

"XCD": 2.70255,

"XDR": 0.727495,

"XOF": 611.957277,

"XPD": 0.00050545,

"XPF": 111.32731,

"XPT": 0.00098188,

"YER": 250.249937,

"ZAR": 15.543525,

"ZMW": 17.033219,

"ZWL": 322

}

}

4.- Retrofit.

Retrofit es una tecnología basada en REST con la cual podremos desarrollar aplicaciones en Android que accedan a servicios web de una forma relativamente sencilla.

Para usar esta tecnología debemos incorporar dos librerías:

- La librería Retrofit para Android y Java (compatible con Kotlin) para hacer llamadas de red y obtener el resultado.
- Una librería que nos permita convertir el resultado devuelto “[parseando](#) ” de forma automática a su objeto; esto facilita mucho realizar peticiones a un API y procesar la respuesta.

Dispone de métodos que manejan API REST con contenidos en formato XML o en formato JSON. Retrofit puede manejar las clásicas primitivas de HTTP (GET, POST, PUT, HEAD...).

El resultado de la consulta al servicio web se incorpora en nuestra aplicación en una estructura de clases que representen los objetos de la respuesta esperada en formato POJO. Estas clases las debemos crear nosotros mismos con la misma estructura que tenga el resultado de la respuesta. Por ejemplo, si la respuesta es en formato JSON, deberemos construir una estructura de clases que represente los campos que nos interesan en la consulta de este JSON.

En este enlace puedes aprender más sobre [el cliente HTTP de Retrofit](#) .

En este otro enlace puedes ver un ejemplo en Kotlin y algunas explicaciones de [cómo obtener datos de Internet desde un cliente Retrofit en una aplicación de Android](#) .

4.1.- Ejercicio resuelto 4 (Retrofit).



Ejercicio Resuelto



(p1104-img001.png)

[Google Developers](https://developer.android.com/studio)

(<https://>

[developer.android.com/](https://developer.android.com/studio)

[studio](https://developer.android.com/studio)).(Uso educativo nc)

Para facilitar la tarea, vamos a tratar de realizar con la tecnología Retrofit un acceso al servicio web de openexchangerates.org del ejercicio visto en el apartado de "Consumo de un Web Service". El ejercicio devolverá el cambio de seis divisas en el momento actual (puedes elegir las que tú quieras) con respecto al dólar, ya que es la información que devuelve el Web Service.

Concretamente, el aspecto que buscaremos para la salida será similar al mostrado en la captura de pantalla de móvil.

En el siguiente vídeo puedes encontrar un ejemplo de cómo construir este tipo de aplicaciones paso a paso con Android Studio.

https://www.youtube.com/embed/Lx1e_fdnNxA (https://www.youtube.com/embed/Lx1e_fdnNxA)

[Descripción textual del vídeo Ejemplo de Retrofit](#)

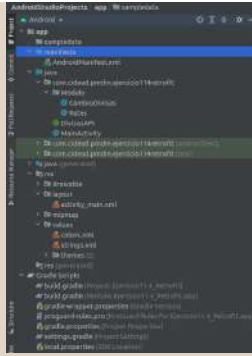
([PMDM11_Descripcion_Video_AndroidJSONRetrofitExample.html](#))



([PMDM11_Descripcion_Video_AndroidJSONRetrofitExample.html](#))

Mostrar retroalimentación

La estructura final de archivos implicados en el proyecto podría ser la siguiente:



(p1104-img002.png)

Google Developers ([https://
developer.android.com/studio](https://developer.android.com/studio))

(Uso educativo nc)

build.gradle (Module app) (#exe-accordion-30-0)

En primer lugar, incorporaremos en **build.gradle (Module app)** las dependencias:

```
1 dependencies {  
2     ...  
3     // Retrofit  
4     implementation "com.squareup.retrofit2:retrofit:2  
5     // Retrofit with Convertes gson  
6     implementation "com.squareup.retrofit2:converter-  
7 }  
}
```

AndroidManifest.xml (#exe-accordion-31-0)

No debemos olvidar configurar el permiso de acceso a Internet en el fichero **AndroidManifest.xml**:

```
1 | <uses-permission android:name="android.permission.I
```

activity_main.xml (#exe-accordion-32-0)

La interfaz es sencilla y quedará definida en el archivo **activity_main.xml**:

```
1 | <?xml version="1.0" encoding="utf-8"?>
2 | <LinearLayout xmlns:android="http://schemas.android
3 |
4 |     xmlns:app="http://schemas.android.com/apk/res-a
5 |
6 |     xmlns:tools="http://schemas.android.com/tools"
7 |
8 |     android:layout_width="match_parent"
9 |
10 |    android:layout_height="match_parent"
11 |
12 |    android:orientation="vertical"
13 |
14 |    tools:context="com.cidead.pmdm.ejercicio114retr
15 |
16 |    <TextView
17 |
18 |        android:id="@+id/tvTitulo"
19 |
20 |        android:layout_width="match_parent"
21 |
22 |        android:layout_height="wrap_content"
23 |
24 |        android:gravity="center"
25 |
26 |        android:text="Acceso a openexchangerates.or
27 |
28 |        android:textSize="20sp"
29 |
30 |        tools:text="API ID"
31 |
32 |        android:layout_margin="15dp"/>
33 |
34 |    <ImageView
35 |
36 |        android:id="@+id/imgvLogin"
37 |
38 |        android:layout_width="match_parent"
```

```
        android:layout_height="80dp"

        app:srcCompat="@drawable/divisas"

        android:layout_margin="20sp"/>

<EditText

    android:id="@+id/etAPIID"

    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    android:hint="Introduce tu API ID"/>

<Button

    android:id="@+id/btnGetData"

    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    android:text="Obtener cambio divisas"

    android:layout_margin="15sp"/>

<TextView

    android:id="@+id/tvBase"

    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    android:gravity="center"

    android:textSize="20sp"

    android:layout_margin="15dp"

/>

<TextView

    android:id="@+id/tvConvert"

    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    android:inputType="textMultiLine"

    android:gravity="center"

    android:textSize="20sp"

    android:layout_margin="15dp"

/>

</LinearLayout>
```

Por otro lado, dentro de la carpeta modelos hemos creado una serie de clases auxiliares que representan el formato de los objetos que vamos a recibir del web service en formato JSON. El formato esperado en el momento de redactar este ejercicio es el mostrado a continuación. Si hubiera cambios tendrás que adaptar los nombres de los atributos de las clases usadas e incluso la estructura de los archivos para que sea similar a la estructura del JSON recibido.

Formato esperado (#exe-accordion-33-0)

```
1  {
2    "disclaimer": "Usage subject to terms: https://op
3    "license": "https://openexchangerates.org/license
4    "timestamp": 1655024402,
5    "base": "USD",
6    "rates": {
7      "AED": 3.6731,
8      "AFN": 89.399125,
9      "ALL": 114.229019,
10     "AMD": 429.910968,
11     "ANG": 1.815174,
12     "AOA": 435.8264,
13     "ARS": 122.446573,
14     "AUD": 1.411943,
15     "AWG": 1.8005,
16     "AZN": 1.7,
17     "BAM": 1.859508,
18     "BBD": 2,
19     "BDT": 93.651536,
20     "BGN": 1.85946,
21     "BHD": 0.376878,
22     "BIF": 2071.295103,
23     "BMD": 1,
24     "BND": 1.391139,
25     ...
26     "ZWL": 322
```

```
}  
}
```

CambioDivisas.java (#exe-accordion-34-0)

Clase CambioDivisas.java:

```
1  package com.cidead.pmdm.ejercicio114retrofit.Modelo
2
3
4  import com.google.gson.annotations.Expose;
5  import com.google.gson.annotations.SerializedName;
6
7
8  public class CambioDivisas {
9
10     @SerializedName("timestamp")
11     @Expose
12     private String timestamp;
13
14     @SerializedName("base")
15     @Expose
16     private String base;
17
18     @SerializedName("rates")
19     @Expose
20     private Rates rates;
21
22
23
24     public String getTimestamp() {
25         return timestamp;
26     }
27
28
29
30     public void setTimestamp(String timestamp) {
31         this.timestamp = timestamp;
32     }
33
34
35
36     public String getBase() {
37         return base;
38     }
39
```

```

    }

    public void setBase(String base) {
        this.base = base;
    }

    public Rates getRates() {
        return rates;
    }

    public void setRates(Rates rates) {
        this.rates = rates;
    }

    @Override
    public String toString() {
        return "CambioDivisas{" +
            "timestamp='" + timestamp + '\'' +
            ", base='" + base + '\'' +
            ", rate=" + rates +
            '}';
    }
}

```

Rates.java (#exe-accordion-35-0)

Clase Rates.java:

```

1 package com.cidead.pmdm.ejercicio114retrofit.Modelo;
2
3
4 import com.google.gson.annotations.Expose;
5 import com.google.gson.annotations.SerializedName;
6
7
8

```



```
public class Rates {  
    @SerializedName("EUR")  
    @Expose  
    private String EUR;  
  
    @SerializedName("AUD")  
    @Expose  
    private String AUD;  
  
    @SerializedName("CNY")  
    @Expose  
    private String CNY;  
  
    @SerializedName("GBP")  
    @Expose  
    private String GBP;  
  
    @SerializedName("ISK")  
    @Expose  
    private String ISK;  
  
    @SerializedName("MXN")  
    @Expose  
    private String MXN;  
  
    @SerializedName("PLN")  
    @Expose  
    private String PLN;  
  
    public String getEUR() {  
        return EUR;  
    }  
  
    public void setEUR(String EUR) {  
        this.EUR = EUR;  
    }  
  
    public String getAUD() {
```

```
        return AUD;
    }

    public void setAUD(String AUD) {
        this.AUD = AUD;
    }

    public String getCNY() {
        return CNY;
    }

    public void setCNY(String CNY) {
        this.CNY = CNY;
    }

    public String getGBP() {
        return GBP;
    }

    public void setGBP(String GBP) {
        this.GBP = GBP;
    }

    public String getISK() {
        return ISK;
    }

    public void setISK(String ISK) {
        this.ISK = ISK;
    }

    public String getMXN() {
        return MXN;
    }

    public void setMXN(String MXN) {
        this.MXN = MXN;
    }
}
```

```

        public String getPLN() {
            return PLN;
        }

        public void setPLN(String PLN) {
            this.PLN = PLN;
        }

        @Override
        public String toString() {
            return "Rates{" +
                "EUR='" + EUR + '\'' +
                ", AUD='" + AUD + '\'' +
                ", CNY='" + CNY + '\'' +
                ", GBP='" + GBP + '\'' +
                ", ISK='" + ISK + '\'' +
                ", MXN='" + MXN + '\'' +
                ", PLN='" + PLN + '\'' +
                '}';
        }
    }
}

```

Configuración de la conexión al servidor web (#exe-accordion-36-0)

Además, para configurar la conexión al servidor web, tenemos la interfaz que conecta con el servicio web que mediante el método GET solicita la aplicación llamada latest.json pasando el parámetro app_id. Si quisiéramos consumir este servicio desde un navegador, tendríamos que usar esta URL con tu ID de API

```
https://openexchangerates.org/api/latest.json?app_id=8b2c865cad6exxxx...
```

, pero con Retrofit debemos construirla de la siguiente manera:

```

1  package com.cidead.pmdm.ejercicio114retrofit;
2
3
4  import com.cidead.pmdm.ejercicio114retrofit.Modelo.
5
6
7  import retrofit2.Call;
8  import retrofit2.http.GET;
9
10 import retrofit2.http.Headers;
11 import retrofit2.http.Query;
12
13
14 public interface DivisasAPI {
15     String BASE_URL = "https://openexchangerates.or
        @Headers("Content-type: application/json")
        @GET("latest.json")
        Call<CambioDivisas> getStuff(@Query("app_id") S
    }

```

MainActivity.java (#exe-accordion-37-0)

Por último la clase MainActivity.java:

```

1  package com.cidead.pmdm.ejercicio114retrofit;
2
3
4  import androidx.appcompat.app.AppCompatActivity;
5
6
7  import android.os.Bundle;
8  import android.util.Log;
9
10 import android.view.View;
11 import android.widget.Button;
12 import android.widget.EditText;
13 import android.widget.TextView;
14 import android.widget.Toast;
15
16
17
18 import com.cidead.pmdm.ejercicio114retrofit.Modelo.
19

```

```

import com.cidead.pmdm.ejercicio114retrofit.Modelo.

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

public class MainActivity extends AppCompatActivity {
    private static final String TAG = "MainActivity";
    private static String BASE_URL = "https://openexchangerates.org/api/";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button btnGetData = (Button) findViewById(R.id.btn_get_data);
        EditText etAPIID = (EditText) findViewById(R.id.et_api_id);
        TextView tvBase = (TextView) findViewById(R.id.tv_base);
        TextView tvConvert = (TextView) findViewById(R.id.tv_convert);

        btnGetData.setOnClickListener(new View.OnClickListener() {

            @Override
            public void onClick(View view) {
                Retrofit retrofit = new Retrofit.Builder()
                    .baseUrl(DivisasAPI.BASE_URL)
                    .addConverterFactory(GsonConverterFactory.create())
                    .build();

                DivisasAPI divisasAPI = retrofit.create(DivisasAPI.class);
                Call<CambioDivisas> call = divisasAPI.getDivisas(etAPIID.getText().toString());

                call.enqueue(new Callback<CambioDivisas>() {

                    @Override
                    public void onResponse(Call<CambioDivisas> call, Response<CambioDivisas> response) {

```

```

        tvBase.setText("Referencia")
        tvConvert.setText("EUR : "
            + "AU"
            + "CN"
            + "IS"
            + "MX"
            + "PL"

        //Trazas
        //Log.d(TAG, "onResponse: R
        //Log.d(TAG, "onResponse: I
    }

    @Override
    public void onFailure(Call<Camb
        //Log.e(TAG, "Error: " + "o
        Toast.makeText(MainActivity
    }

    });
}

});
}

}
}

```

5.- Obtención de imágenes a través de la web.

Existen varias librerías que permiten descargarnos imágenes procedentes de servidores web. Las más conocidas son Picasso, Glide, Universal Image Loader y Fresco. Además, incluso Retrofit dispone de métodos relacionados con la carga de imágenes como `getImage()`.

La librería Picasso se caracteriza por su sencillez y poco código necesario para poder cargar imágenes con buena calidad (por ejemplo la calidad de imagen que maneja Glide es algo peor, pero a diferencia de Picasso, Glide soporta formato GIF animados). Será necesario incluir la librería a través de la inclusión en el apartado de dependencias del `build.gradle` (Module: app) como hemos hecho con otras librerías, en el apartado de las dependencias.



Para saber más

Aquí puedes encontrar más detalles sobre la [librería Picasso](#) .

5.1.- Ejercicio resuelto 5 (Imágenes con Picasso).



Ejercicio Resuelto

Se pide, usando la librería Picasso, conectar con una API de la NASA que publica imágenes de Marte capturadas por el Rover Curiosity.

NOTA: Para acceder a esta imagen no es necesario usar una API_key aunque puedes conseguirla registrándote con tu correo electrónico.

Las APIs están publicadas en esta página web: <https://api.nasa.gov/>.

Como puedes ver en la figura 1, debes ir a la página, pulsar el botón de la parte superior de "Browse APIs" y, a continuación, desplazarnos hasta abajo y encontrar los detalles de la API "Mars Rover Photos".

En la figura 2 vemos que pulsando al primer enlace (https://api.nasa.gov/mars-photos/api/v1/rovers/curiosity/photos?sol=1000&api_key=DEMO_KEY) de "Example Queries" podemos ver la estructura del JSON devuelto por el servicio.

Desarrolla una App Android que muestre, mediante la librería Picasso, tres de estas imágenes servidas desde estas páginas. Para ello puedes conseguir los enlaces mostrados en alguna de las imágenes publicadas del JSON. El aspecto de la aplicación debe ser el mostrado en la figura 3.

Figura 1



([p1105-img002.png](#))

Google Developers (<https://developer.android.com/studio>)

(Uso educativo nc)

Figura 2



([p1105-img003.png](#))

🔍 [Google Developers \(https://
developer.android.com/studio\)](https://developer.android.com/studio) (Uso educativo
nc)

Figura 3



(p1105-img001.png)

🔍 [Google Developers
\(https://
developer.android.com/
studio\)](https://developer.android.com/studio) (Uso educativo nc)

Mostrar retroalimentación

AndroidManifest.xml (#exe-accordion-38-0)

Añadimos el permiso al **AndroidManifest.xml**:

```
1 | <uses-permission android:name="android.permission.I
```

build.gradle(:app) (#exe-accordion-39-0)

En el archivo build.gradle(:app) añadimos la librería Picasso

en el apartado de las dependencias:

```
1 dependencies {
2
3     implementation 'androidx.appcompat:appcompat:1.
4     ...
5     //Añadimos la librería Picasso
6     implementation 'com.squareup.picasso:picasso:2.
7
8 }
```

activity_main.xml (#exe-accordion-40-0)

La interfaz vendrá dada por **activity_main.xml**:

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
3     xmlns:app="http://schemas.android.com/apk/res-a
4     xmlns:tools="http://schemas.android.com/tools"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     tools:context=".MainActivity">
8
9     <ImageView
10         android:id="@+id/iv1"
11         android:layout_width="162dp"
12         android:layout_height="125dp"
13         tools:layout_constraintTop_creator="1"
14         android:layout_marginStart="8dp"
15         android:layout_marginTop="8dp"
16         tools:layout_constraintLeft_creator="1"
17         app:layout_constraintLeft_toLeftOf="parent"
18         app:layout_constraintTop_toTopOf="parent"
19         android:layout_marginLeft="8dp" />
20
21
22
23
24
25
26
27
28
```

```
<TextView
    android:id="@+id/tv1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Nasa - Curiosity - Img0"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintHorizontal_bias="0.725"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.081"
```

```
<ImageView
    android:id="@+id/iv2"
    android:layout_width="162dp"
    android:layout_height="125dp"
    android:layout_marginLeft="8dp"
    android:layout_marginStart="8dp"
    android:layout_marginTop="180dp"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    tools:layout_constraintLeft_creator="1"
    tools:layout_constraintTop_creator="1" />
```

```
<TextView
    android:id="@+id/tv2"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Nasa - Curiosity - Img10"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintHorizontal_bias="0.748"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.324"
```

```
<ImageView
```

```

        android:id="@+id/iv3"
        android:layout_width="162dp"
        android:layout_height="125dp"
        android:layout_marginLeft="8dp"
        android:layout_marginStart="8dp"
        android:layout_marginTop="337dp"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        tools:layout_constraintLeft_creator="1"
        tools:layout_constraintTop_creator="1" />

<TextView
    android:id="@+id/tv3"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Nasa - Curiosity - Img20"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintHorizontal_bias="0.748"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.561"

</androidx.constraintlayout.widget.ConstraintLayout>

```

MainActivity.java (#exe-accordion-41-0)

Finalmente, el archivo **MainActivity.java**:

```

1 | package com.cidead.pdm.ejercicio115;
2 |
3 |
4 | import androidx.appcompat.app.AppCompatActivity;
5 |
6 |
7 |

```

```
import android.os.Bundle;

import android.widget.ImageView;

import com.squareup.picasso.Picasso;

public class MainActivity extends AppCompatActivity {

    ImageView iv1;

    ImageView iv2;

    ImageView iv3;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        iv1=(ImageView)findViewById((R.id.iv1));
        iv2=(ImageView)findViewById((R.id.iv2));
        iv3=(ImageView)findViewById((R.id.iv3));
        //URL de la API https://api.nasa.gov/mars-p
        Picasso.get()
            .load("https://mars.nasa.gov/msl-ra
            .resize(500, 500)
            .centerCrop()
            .into(iv1);
        Picasso.get()
            .load("https://mars.nasa.gov/msl-ra
            .resize(500, 500)
            .centerCrop()
            .into(iv2);
        Picasso.get()
            .load("https://mars.nasa.gov/msl-ra
            .resize(500, 500)
            .centerCrop()
            .into(iv3);
        /* Otra forma de llamar aun mas corta
        Picasso.get().load("https://mars.nasa.gov/m
    }
}
```

}

IV - Geolocalización y mapas de Google.



Caso práctico



Q [cottonbro \(https://www.pexels.com/es-es/foto/manos-iphone-telefono-movil-navegacion-4542998/\)](https://www.pexels.com/es-es/foto/manos-iphone-telefono-movil-navegacion-4542998/).(Pexels)

El cliente de BK Programación que solicitó la aplicación para sus compradores de bicicletas, que tomaba valores a partir de los sensores de los dispositivos Android, desea incorporar algunas funcionalidades más relacionadas con el tracking de rutas.

- ¿Os acordáis del cliente que vendía bicicletas? - pregunta Ada.

- ¡Cómo no! -responde Pedro-, aprendimos un montón sobre sensores con ese pedido.

- Ahora quiere incorporar el tracking de rutas -informa Ada-, y creo que lo más adecuado es utilizar los mapas y la tecnología de Google Maps.

Incorporar esta tecnología supone usar la API **Maps SDK for Android** que Google ofrece a los desarrolladores y a las empresas de forma gratuita hasta un umbral o número de veces en que el mapa se muestra a los usuarios finales. Si mensualmente se supera este umbral, Google realizará cobros por entender que los usuarios o los intermediarios usan la aplicación Google Maps de forma comercial.

- Comenzaremos registrando una cuenta en **Google Cloud** -explica Ada-, y configurando un proyecto donde activemos la **Maps SDK for Android** para desarrolladores de Android. Sólo queda aprender cómo crear fragmentos, actividades y elementos de la pantalla que sean capaces de interactuar con la aplicación y apoyarse en el servicio de mapas de Google.

-¡Comencemos! -exclama el equipo al unísono.



  [Ministerio de Educación y Formación Profesional.](#)

(Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

 [Aviso Legal](#) 

1.- Geolocalización.



Caso práctico



- Estamos viendo -informa Juan a Ada- que para realizar un tracking de rutas es necesario hacer geolocalizaciones.
- Muy bien visto -felicitada Ada-, y ¿qué ocurre cuando una aplicación quiere acceder a nuestra ubicación?
- Que debemos darle permisos -responde Juan-. Vamos a investigar sobre las características de esos permisos.

Q Caleb Oquendo
([https://
www.pexels.com/es-es/
foto/telefono-
inteligente-direccion-
telefono-movil-
navegacion-7738879/](https://www.pexels.com/es-es/foto/telefono-inteligente-direccion-telefono-movil-navegacion-7738879/)).(Pexels)

Una de las funciones importantes de las aplicaciones móviles es la detección de ubicaciones. Los usuarios de plataformas móviles llevan consigo sus dispositivos a todas partes. A su vez, cuando se agrega la detección de ubicaciones se ofrece a los usuarios una experiencia más contextual.

Sin embargo, como desarrolladores de aplicaciones debemos ser conscientes de la importancia de la privacidad de los datos de nuestros usuarios. En este elemento concreto, la ubicación de los usuarios puede ser un elemento muy importante a proteger, por ello las aplicaciones que usan servicios de ubicación deben solicitar permisos para ello. Por este motivo debemos solicitar a nuestros usuarios que nos conceda el permiso adecuado que necesitemos.

Cada permiso tiene una combinación de dos características, la **categoría** o modo en que se ejecuta el acceso a la ubicación (en **primer plano** o en **segundo plano**) y la **precisión** de la ubicación (precisión **aproximada** o **precisa**).

1.1.- Tipos de acceso a la ubicación.

Categoría ubicación en primer plano

Se usa cuando la ubicación es necesaria en algún momento concreto. Por ejemplo, cuando una aplicación de mensajería necesita compartir la ubicación del usuario. En este caso es necesario declarar en el archivo de manifiesto una entrada de servicio indicando "location".

Código

```
1 <service
2     android:name="MyNavigationService"
3     android:foregroundServiceType="location" ... >
4
5     <!-- Any inner elements would go here. -->
6
7 </service>
```

El sistema considera que la aplicación usa esta categoría si:

- Una Activity de la aplicación que usa "ubicación" es visible.
- La aplicación ejecuta un servicio en primer plano que se mantiene activo incluso cuando la aplicación no es la que está como principal, o incluso la pantalla está apagada.

En cuanto a la precisión, es posible especificar si necesitamos ubicación precisa o ubicación predeterminada. En ambos casos, en el archivo de manifiesto deberemos indicar cuál de los permisos es necesario.

Categoría ubicación en segundo plano

Una aplicación requiere acceso a la ubicación en segundo plano si una función dentro de la aplicación comparte constantemente la ubicación con otros usuarios o usa la [API de geovallado](#) . Por ejemplo, una aplicación familiar donde se comparte la ubicación de alguno de los miembros con otros.

El sistema considera que tu aplicación usa ubicación en segundo plano si hace un uso diferente al indicado en primer plano. Para poder usar este tipo de aplicación es necesario indicar en el archivo de manifiesto el siguiente permiso:

Código

```
1 <manifest ... >
2     <!-- Required only when requesting background location access on
3
```

```
Android 10 (API level 29) and higher. -->
```

```
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
</manifest>
```



Debes conocer

Google Play Store tiene una [política sobre la ubicación del dispositivo](#) en la que el acceso a la ubicación en segundo plano se restringe a las aplicaciones que lo necesiten para su funcionalidad principal, y cumplan los requisitos de políticas relacionadas.

1.2.- Tipos de precisión.

Android admite los siguientes niveles de precisión de la ubicación:

Aproximada

Proporciona una estimación de la ubicación del dispositivo. Si esta estimación de la ubicación es de `LocationManagerService` o `FusedLocationProvider`, la estimación es exacta en un radio de 3 kilómetros cuadrados. Tu aplicación puede recibir ubicaciones con este nivel de precisión cuando declaras el permiso `ACCESS_COARSE_LOCATION`, pero no el permiso `ACCESS_FINE_LOCATION`. Android usa datos de Wi-Fi o datos móviles para determinar la ubicación del dispositivo.

Precisa

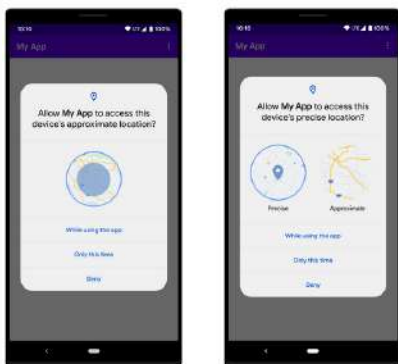
Proporciona una estimación de la ubicación del dispositivo que sea lo más precisa posible. Si la estimación de la ubicación es de `LocationManagerService` o `FusedLocationProvider`, suele ser de 50 metros y, a veces, tan precisa como de unos pocos metros. Tu aplicación puede recibir ubicaciones en este nivel de precisión cuando declaras el permiso `ACCESS_FINE_LOCATION`. Android usa el sistema de posicionamiento global (GPS) y los datos de Wi-Fi y los datos móviles para determinar la ubicación del dispositivo.

El permiso que elijamos determina la precisión de la ubicación devuelta por la API. Según el nivel de precisión que necesitemos, solo debes solicitar uno de los permisos de ubicación de Android en el archivo de manifiesto.

Ejemplo

```
1  <manifest ... >
2      <!-- Always include this permission -->
3
4      <uses-permission android:name="android.permission.ACCESS_COARSE_LO
5
6
7      <!-- Include only if your app benefits from precise location acce
          <uses-permission android:name="android.permission.ACCESS_FINE_LOCA
          </manifest>
```

1.3.- Solicitud de permisos en tiempo de ejecución.



Q [Google Developers](#) (Uso educativo nc)

Cuando una función de tu aplicación necesite acceso a la ubicación, espera hasta que el usuario interactúe con ella para solicitar el permiso.

En Android 12 (nivel de API 31) o versiones posteriores, los usuarios pueden solicitar que tu aplicación solo recupere información de ubicación aproximada, incluso cuando tu aplicación solicite el permiso de ubicación precisa en tiempo de ejecución. Se recomienda en general solicitar sólo la ubicación aproximada.

Para ofrecer esta elección al usuario, la aplicación debe solicitar en el archivo de manifiesto `ACCESS_FINE_LOCATION` y `ACCESS_COARSE_LOCATION`. En este caso, el diálogo de permisos del sistema incluirá ambas opciones para el usuario, como puede verse en la segunda captura de la imagen de esta página.

Observa que, además, el usuario puede elegir si desea otorgar el permiso mientras la aplicación está en uso (siempre que esté en uso), sólo para esta ocasión, o denegarlo.



Autoevaluación

El sistema considera que la aplicación usa la ubicación en primer plano cuando...

- ☐ La aplicación ejecuta un servicio en primer plano que se mantiene activo incluso cuando la aplicación no es la que está como principal o incluso la pantalla está apagada.
- ☐ Una función dentro de la aplicación comparte constantemente la ubicación con otros usuarios.
- ☐ Una Activity de la aplicación que usa "ubicación" es visible.

Mostrar retroalimentación

Solución

1. Correcto (#answer-369_139)

2. Incorrecto (#answer-369_11169)

3. Correcto (#answer-369_11171)

2.- Mapas de Google.



Caso práctico

- Ya tenemos claro el funcionamiento de los permisos para que una aplicación pueda realizar geolocalizaciones -informa María a Ada.
- ¡Genial! -exclama Ada-. Ahora vamos a realizar una pequeña práctica con la tecnología de Google Maps para afianzar esos conocimientos.



🔍 Brett Jordan (<https://www.pexels.com/es-es/foto/iphone-telefono-inteligente-aplicacion-ios-5444631/>) (Pexels)

Con la API **Maps SDK for Android**, puedes agregar mapas a tu aplicación para Android, incluidas las aplicaciones para Wear OS que utilizan datos, reproducciones de mapas y respuestas gestuales de Google Maps. También puedes ofrecer información adicional sobre las ubicaciones del mapa y facilitar la interacción con el usuario si agregas marcadores, polígonos y superposiciones a tu mapa.

El SDK es compatible con los lenguajes de programación Kotlin y Java, y ofrece bibliotecas y extensiones adicionales para funciones avanzadas como agregar:

- Iconos anclados en posiciones específicas del mapa (veremos en el ejercicio resuelto el uso del método ***addMarker()*** para crear marcadores en el mapa).
- Conjuntos de segmentos de líneas (polilíneas).
- Segmentos cerrados (polígonos).
- Gráficos de mapa de bits anclados en posiciones específicas del mapa (marcadores).
- Conjuntos de imágenes que se muestran sobre los mosaicos de mapas básicos (superposiciones de mosaicos).

2.1.- Ejercicio resuelto 1 (mapas de Google Maps y geolocalización).



Debes conocer

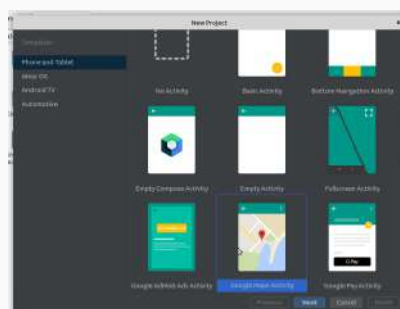
Para llevar a cabo este ejercicio es necesario que consigas una "API Key" que te permita interactuar con los servicios de Google. Para ello es necesario contar con una cuenta de **correo de Google y una tarjeta de crédito válida**.

Todos los ejercicios se pueden llevar a cabo sin coste, ya que Google, en el momento de escribir este contenido, ofrece un periodo de prueba gratuito de 90 días o 300\$. Este crédito es más que suficiente para llevar a cabo los ejercicios propuestos. Esta cuenta de prueba puede cancelarse una vez transcurrido el tiempo de la realización de estas prácticas. Digamos que, para poder activar la API Key Google, comprueba que el proyecto está asociado a una cuenta de facturación correcta, aunque son necesarias miles de visitas para que se cobre por ello. Esto está orientado para empresas que se apoyan en el servicio de Google Maps para ofrecer sus servicios.

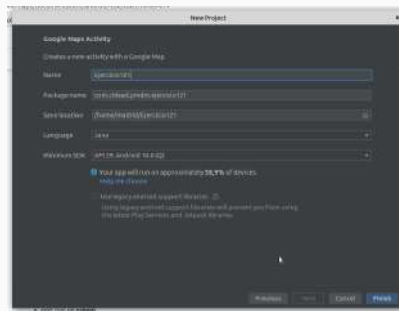
En caso de no disponer de tarjeta de crédito, debes contactar con el tutor del curso para salvar este problema, probablemente él pueda compartir una APY Key con propósitos educativos.

Se trata de crear una aplicación en la que agregues un mapa mediante el **Maps SDK for Android** y lo muestres con una ubicación. Como EXTRA trabajaremos para que muestre la ubicación actual de tu terminal.

1.- El primer paso es crear un proyecto de Google Maps en Android Studio ([más ayuda](#)



Q [Google Developers](#) (Uso educativo no)



Q [Google Developers](#) (Uso educativo nc)

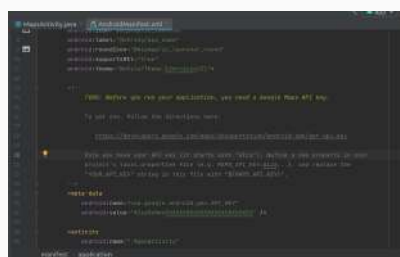
Esta acción terminará habiendo creado el proyecto y mostrará dos ficheros abiertos sobre los que trabajar, el archivo de manifiesto y **MapsActivity.java**. Si observas, se ha creado un archivo **activity_maps.xml** donde se despliega un fragmento para albergar el mapa de Google a pantalla completa.

2. El segundo paso consiste en conseguir una API Key de Google Maps. Para ello tendrás que registrar tu cuenta de correo en [Cloud console de Google](#), crear un proyecto nuevo, crear una cuenta de facturación donde asignes datos de facturación (incluso una tarjeta de crédito obligatoria) y finalmente activar la API **Maps SDK for Android**. Es posible que el nombre del servicio y la forma de hacerlo varíe en el tiempo. Deberás encontrar información de cómo hacerlo en Internet. En el momento de escribir este contenido, puedes encontrar ayuda de cómo hacerlo en este [enlace](#).

3. El siguiente paso es añadir nuestra API Key al proyecto. Para ello lo más sencillo es seguir las indicaciones de Android Studio y completar la variable android:value del elemento <meta-data> con tu API key (figura 1).

El resultado es una aplicación que abre el fragmento de Google Maps con el mapa ubicado en Sidney (Australia), como se muestra en la figura 2.

Figura 1



Q [Google Developers](#) (Uso educativo nc)

Figura 2



Q [Google Developers](#)

(Uso educativo nc)

Si observas el código generado automáticamente en el archivo **MapsActivity.java**, podrás observar que se ha programado el evento **onMapReady()** para que, cuando esté preparado, se actualice y se añada un marcador (la tachuela roja de siempre) mediante la función **addMarker()**. Finalmente, se centra la posición del mapa en el marcador mediante el método **moveCamera()** pasando el objeto que guarda las coordenadas. También es posible cambiar el aspecto del marcador.

MapsActivity.java

```
1 | @Override
2 | public void onMapReady(GoogleMap googleMap) {
3 |
4 |     mMap = googleMap;
5 |
6 |     // Add a marker in Sydney and move the camera
7 |     LatLng sydney = new LatLng(-34, 151);
8 |     mMap.addMarker(new MarkerOptions().position(sydney).title("Marker in Sydney"));
9 |     mMap.moveCamera(CameraUpdateFactory.newLatLng(sydney));
   | }

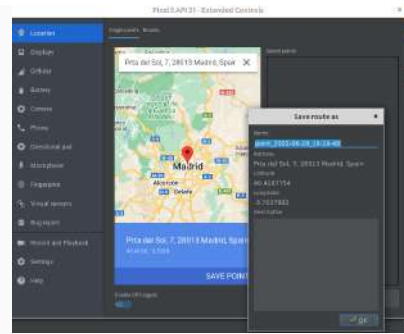
```

Elije unas coordenadas de latitud y longitud de un lugar que te guste (tu pueblo, tu ciudad, un sitio que te guste) y actualiza la línea de código dónde se crea el objeto de la clase `LatLng`. Prueba de nuevo la aplicación y verifica que el mapa se abre las coordenadas indicadas.

```
1 | LatLng sydney = new LatLng(Latitud elegida, Longitud elegida);

```

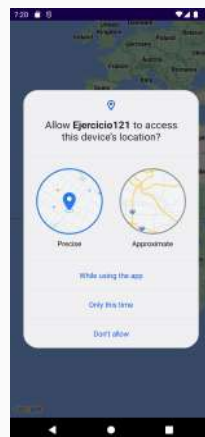
4. APARTADO EXTRA: El último paso consiste en cambiar esta ubicación por defecto para que en realidad el mapa muestre nuestra ubicación. Puedes establecer la ubicación para los terminales emulados mediante la opción "Location" de los controles extendidos.



Q [Google Developers](#) (Uso educativo nc)

Te recomendamos que, después de establecer la localización mediante los controles extendidos, y antes de ejecutar la aplicación, abras Google Maps en el emulador y verifiques que te reconoce la ubicación que hayas establecido como tu ubicación actual. De esta forma te aseguras que el terminal ya ha solicitado la ubicación y tiene algún valor al respecto.

Ahora, al ejecutar la aplicación te solicitará los permisos y, finalmente, mostrará la ubicación elegida (en este caso, Madrid).



Q [Google Developers](#) (Uso educativo nc)



Q [Google Developers](#) (Uso educativo nc)

A continuación te mostramos el contenido de algunos archivos importantes.

AndroidManifest.xml

Permisos en **AndroidManifest.xml**.

```
1 | <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
2 | <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
3 |
4 | <uses-permission android:name="android.permission.INTERNET" />
   |
   | <uses-permission android:name="android.permission.ACCESS_BACKGROUND_TASKS" />
```

build.gradle(:app)

Dependencias en **build.gradle(:app)**.

```
1 | dependencies {
2 |
3 |     implementation 'androidx.appcompat:appcompat:1.4.2'
4 |     implementation 'com.google.android.material:material:1.6.1'
5 |     implementation 'com.google.android.gms:play-services-maps:18.0.0'
6 |     implementation 'com.google.android.gms:play-services-location:21.0.0'
7 |     implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
8 |     implementation 'org.jetbrains:annotations:16.0.2'
9 |
10 |     testImplementation 'junit:junit:4.13.2'
11 |     androidTestImplementation 'androidx.test.ext:junit:1.1.3'
12 |     androidTestImplementation 'androidx.test.espresso:espresso-core:3.4.0'
13 |
14 | }
```

MapsActivity.java

Archivo **MapsActivity.java**.

```
1 | package com.cidead.pmdm.ubicacion;
2 |
3 |
4 | import androidx.annotation.NonNull;
```

```

import androidx.core.app.ActivityCompat;
import androidx.fragment.app.FragmentActivity;

import android.Manifest;
import android.content.pm.PackageManager;
import android.location.Location;
import android.os.Bundle;
import android.widget.Toast;

import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationServices;
import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.SupportMapFragment;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.MarkerOptions;
import com.cidead.pdm.ubicacion.databinding.ActivityMapsBinding;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.android.gms.tasks.Task;

public class MapsActivity extends FragmentActivity implements OnMapReadyCallback {

    private GoogleMap mMap;
    private ActivityMapsBinding binding;
    Location currentLocation;
    FusedLocationProviderClient fusedLocationProviderClient;
    private static final int REQUEST_CODE=101;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        binding = ActivityMapsBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());
        fusedLocationProviderClient = LocationServices.getFusedLocationProviderClient(this);
        getCurrentLocation();
    }

```

```

    }

    /**
     * Manipulates the map once available.
     * This callback is triggered when the map is ready to be used.
     * This is where we can add markers or lines, add listeners or
     * we just add a marker near Sydney, Australia.
     * If Google Play services is not installed on the device, the
     * it inside the SupportMapFragment. This method will only be
     * installed Google Play services and returned to the app.
     */
    @Override
    public void onMapReady(GoogleMap googleMap) {
        mMap = googleMap;

        LatLng tuUbicacion = new LatLng(currentLocation.getLatitude(),
            currentLocation.getLongitude());
        mMap.addMarker(new MarkerOptions().position(tuUbicacion).title("Ubicación"));
        mMap.moveCamera(CameraUpdateFactory.newLatLng(tuUbicacion));
    }

    private void getCurrentLocation(){
        if (ActivityCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED && ActivityCompat.checkSelfPermission(this, Manifest.permission.ACCESS_COARSE_LOCATION) != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this, new String[]{Manifest.permission.ACCESS_FINE_LOCATION}, 1);
            return;
        }
        Task<Location> task = fusedLocationProviderClient.getLastLocation();
        task.addOnSuccessListener(new OnSuccessListener<Location>() {
            @Override
            public void onSuccess(@NonNull @org.jetbrains.annotations.NotNull Location location) {
                if (location != null) {
                    currentLocation = location;
                    Toast.makeText(MapsActivity.this, String.valueOf(currentLocation.getLatitude() + " " + currentLocation.getLongitude()), Toast.LENGTH_SHORT).show();
                    SupportMapFragment supportMapFragment = (SupportMapFragment) getSupportFragmentManager().findFragmentById(R.id.map);
                    assert supportMapFragment != null;
                    supportMapFragment.getMapAsync(MapsActivity.this);
                }
            }
        });
    }
}

```

```

    });

}

@Override
public void onRequestPermissionsResult(int requestCode, @NonNull
    super.onRequestPermissionsResult(requestCode, permissions,
    switch (REQUEST_CODE) {
        case REQUEST_CODE:
            if (grantResults.length > 0 && grantResults[0] == P
                getLocation();
            }
            break;
        }
    }
}

```

Es una tarea compleja. Puedes apoyarte en este vídeo para llevarlo a cabo:

<https://www.youtube.com/embed/cnlSyYeRqrs>



Descripción textual del vídeo "Localización actual"



Puedes consultar más ayuda en esta guía rápida de creación de aplicaciones de [Google Maps para Android Studio](#).



Para saber más

Google recomienda por seguridad otras opciones para añadir la API Key a la aplicación mediante local.properties. Aquí tienes [más ayuda para intentar esta otra opción](#) para intentarlo.

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0 (<http://creativecommons.org/licenses/by-sa/4.0/>).